

SKRIPSI

IMPLEMENTASI ALGORITMA 1D-CNN UNTUK APLIKASI KLASIFIKASI ANOMALI JALAN MENGGUNAKAN DATA SENSOR IMU SMARTPHONE

Diajukan untuk memenuhi salah satu syarat
memperoleh gelar Sarjana Komputer



Disusun Oleh:

Nama : Muhammad Rizki
NIM : A11.2022.14657
Program Studi : Teknik Informatika

FAKULTAS ILMU KOMPUTER
UNIVERSITAS DIAN NUSWANTORO
SEMARANG
2026

PERSETUJUAN SKRIPSI

Nama : Muhammad Rizki
NIM : A11.2022.14657
Program Studi : Teknik Informatika
Fakultas : Ilmu Komputer
Judul Tugas Akhir : Implementasi Algoritma 1D-CNN Untuk Aplikasi
Klasifikasi Anomali Jalan Menggunakan Data Sensor
IMU Smartphone

Tugas Akhir ini telah diperiksa dan disetujui,
Semarang, <tanggal *format DD MM YYYY*>

Menyetujui:
Pembimbing

Mengetahui:
Dekan Fakultas Ilmu Komputer

Egia Rosi Subhiyakto, M.Kom

Dr. Abdul Syukur

PENGESAHAN DEWAN PENGUJI

Nama : Muhammad Rizki
NIM : A11.2022.14657
Program Studi : Teknik Informatika
Fakultas : Ilmu Komputer
Judul Tugas Akhir : Implementasi Algoritma 1D-CNN Untuk Aplikasi
Klasifikasi Anomali Jalan Menggunakan Data Sensor
IMU Smartphone

Tugas akhir ini telah diujikan dan dipertahankan dihadapan Dewan Penguji pada
Sidang tugas akhir tanggal Menurut pandangan kami, tugas akhir ini
memadai dari segi kualitas maupun kuantitas untuk tujuan penganugerahan
gelar Sarjana Komputer (S.Kom)

Semarang, <tanggal *format DD MM YYYY*>

Dewan Penguji:

Adhitya Nugraha, S.Kom, M.CS
Anggota Penguji 1

Ardytha Luthfiarta, M.Kom
Anggota Penguji 2

Sri Winarno, Ph.D
Ketua Penguji

PERNYATAAN KEASLIAN SKRIPSI

Sebagai mahasiswa Universitas Dian Nuswantoro, yang bertanda tangan dibawah ini, saya:

Nama : Muhammad Rizki

NIM : A11.2022.14657

Menyatakan bahwa karya ilmiah saya yang berjudul:

Implementasi Algoritma 1D-CNN Untuk Aplikasi Klasifikasi Anomali Jalan Menggunakan Data Sensor IMU Smartphone.

Merupakan karya asli saya (kecuali cuplikan dan ringkasan yang masing – masing telah saya jelaskan sumbernya). Apabila di kemudian hari, karya saya disinyalir bukan merupakan karya asli saya, yang disertai dengan bukti – bukti yang cukup, maka saya bersedia untuk dibatalkan gelar saya beserta hak dan kewajiban yang melekat pada gelar tersebut. Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Semarang

Pada tanggal : *<format DD MM YYYY>*

Yang menyatakan

(.....)

UCAPAN TERIMAKASIH

Dengan memanjatkan puji syukur kehadiran Allah SWT Tuhan Yang Maha Pengasih dan Maha Penyayang yang telah melimpahkan segala Rahmat, hidayah dan inayah-Nya kepada penulis sehingga laporan tugas akhir dengan judul:

“Implementasi Algoritma 1D-CNN Untuk Aplikasi Klasifikasi Anomali Jalan Menggunakan Data Sensor IMU Smartphone“

dapat penulis selesaikan dengan rencana karena dukungan dari berbagai pihak. Oleh karena itu, penulis ingin menyampaikan ucapan terimakasih kepada:

1. Prof. Dr. Ir. Edi Noersasongko, M.Kom selaku Rektor Universitas Dian Nuswantoro Semarang.
2. Dr. Abdul Syukur selaku Dekan Fakultas Ilmu Komputer.
3. Sri Winarno, Ph.D selaku Ketua Program Studi Teknik Informatika.
4. Egia Rosi Subhiyakto, M.Kom selaku dosen pembimbing tugas akhir yang memberikan ide penelitian, informasi referensi dan bimbingan yang berkaitan dengan penelitian penulis.
5. Dosen – dosen pengampu di Fakultas Ilmu Komputer Teknik Informatika Universitas Dian Nuswantoro Semarang yang telah memberikan ilmu dan pengalamannya masing – masing sehingga penulis dapat mengimplementasikan ilmu yang telah diberikan,
6. Kedua orang tua tercinta yang senantiasa memberikan doa, kasih sayang, dukungan, motivasi, serta bimbingan kepada penulis dalam setiap langkah kehidupan, sehingga penulis dapat menyelesaikan penyusunan skripsi ini.
7. Teman-teman grup "Pejuang Sarjana", yaitu Wildan, Alep, Mupti, Wpi, dan Ita, yang telah menjadi rekan seperjuangan sejak masa SMA serta senantiasa memberikan doa, dukungan, motivasi, dan semangat kepada penulis, baik secara eksplisit maupun implisit, sehingga penulis dapat menyelesaikan penyusunan skripsi ini.
8. Teman-teman komunitas Discord, yaitu Aya, Shey, Isal, Aziz, Jamal, Hadid, Eneng, serta teman-teman lain yang tidak dapat disebutkan satu per satu, yang telah memberikan dukungan, motivasi, semangat, serta menjadi tempat berbagi

cerita selama proses penyusunan skripsi ini sehingga penulis dapat menyelesaikan tanggung jawab akademiknya.

Semoga Tuhan Yang Maha Esa memberikan balasan yang lebih besar kepada beliau – beliau dan pada akhirnya penulis berharap bahwa penulisan laporan tugas akhir ini dapat bermanfaat dan berguna sebagaimana fungsinya.

Semarang, Agustus 2024

Penulis

ABSTRAK

Pemeliharaan infrastruktur jalan beraspal sering terkendala oleh pelaporan manual yang lambat, sedangkan pendekatan berbasis cloud IoT dibatasi oleh latensi jaringan dan konsumsi daya transmisi. Penelitian ini mengusulkan arsitektur Edge-Centric berupa aplikasi Android yang mengakuisisi dan mengklasifikasikan anomali permukaan jalan (*pothole* dan *speed bump*) secara langsung pada perangkat menggunakan sensor Inertial Measurement Unit (IMU) smartphone. Model Lightweight InceptionTime 1D-CNN multi-branch dengan Squeeze-and-Excitation Block digunakan untuk mengekstraksi fitur spasio-temporal dari data sensor mentah 7 kanal berfrekuensi 100 Hz tanpa rekayasa fitur manual. Untuk meningkatkan robustitas model diterapkan Global Z-Score Normalization serta Multi-Class Focal Loss guna mengatasi ketidakseimbangan kelas. Evaluasi menggunakan Trip-Based Holdout Split (80:20) sehingga data pengujian berasal dari lintasan yang berbeda dengan data pelatihan. Model mencapai Macro F1-Score 83% dengan recall pothole 62% pada holdout test set. Model kemudian dikonversi ke format ONNX dan diintegrasikan ke aplikasi Android melalui ONNX Runtime Mobile sehingga mendukung inferensi real-time secara offline. Hasil penelitian menunjukkan bahwa kombinasi Edge AI dan smartphone berpotensi mendukung pemetaan kerusakan jalan secara real-time tanpa ketergantungan pada koneksi internet.

Kata Kunci: 1D-CNN, Deteksi Lubang, Akselerometer, ONNX

ABSTRACT

Road infrastructure maintenance is often hindered by slow manual reporting, while cloud IoT-based approaches are limited by network latency and continuous transmission power consumption. This study proposes an Edge-Centric Android application capable of acquiring and classifying road surface anomalies (*potholes* and *speed bumps*) directly on the device using the smartphone's Inertial Measurement Unit (IMU) sensors. A Lightweight InceptionTime 1D-CNN with a multi-branch architecture and Squeeze-and-Excitation Block was employed to extract spatio-temporal features from seven-channel raw sensor data sampled at 100 Hz without manual feature engineering. To improve model robustness, Global Z-Score Normalization and Multi-Class Focal Loss were applied to address significant class imbalance. Model evaluation was conducted using an 80:20 Trip-Based Holdout Split, ensuring that the test data originated from different travel trajectories than the training data. The proposed model achieved a Macro F1-Score of 83% with a pothole recall of 62% on the holdout test set. The trained model was subsequently converted to the ONNX format and deployed on Android using ONNX Runtime Mobile, enabling real-time offline inference. These results demonstrate that integrating Edge AI with mid-range smartphones offers a practical solution for real-time road damage mapping without relying on an internet connection.

Keywords: 1D-CNN, Pothole Detection, Accelerometer, ONNX, Edge AI

DAFTAR ISI

PERSETUJUAN SKRIPSI	ii
PENGESAHAN DEWAN PENGUJI	iii
PERNYATAAN KEASLIAN SKRIPSI	iv
UCAPAN TERIMAKASIH	v
ABSTRAK	vii
DAFTAR ISI	ix
DAFTAR GAMBAR	xii
DAFTAR LAMPIRAN	xiii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Batasan Masalah	3
1.4 Tujuan Penelitian	4
1.5 Manfaat Penelitian	4
1.5.1. Manfaat Akademis	4
1.5.2. Manfaat Praktis	5
BAB II LANDASAN TEORI	6
2.1 Tinjauan Studi	6
2.2 Tinjauan Pustaka	12
2.2.1. Anomali Permukaan Jalan Beraspal	12
2.2.2. Sensor Smartphone (IMU dan GPS)	13
2.2.3. <i>Deep Learning</i>	14
2.2.4. <i>One-Dimensional Convolutional Neural Network (1D-CNN)</i> ..	15
2.2.5. Edge Computing dan ONNX Runtime	17
2.2.6. Segmentasi Data Runtun Waktu (<i>Time-Series Windowing</i>)	17
2.2.7. Evaluation Metrics	19

2.3 Kerangka Pemikiran.....	22
BAB III METODE PENELITIAN.....	23
3.1 Alur Penelitian.....	23
3.2 Desain Penelitian.....	25
3.2.1. Peralatan Penelitian.....	25
3.2.2. Kebutuhan Data.....	25
3.3 Akuisisi dan Pra-Pemrosesan Data.....	26
3.3.1. Pengembangan Aplikasi Data Logger.....	26
3.3.2. Akuisisi Data Lapangan	26
3.3.3. Pelabelan Data.....	28
3.3.4. Pra-Pemrosesan Data	29
3.4 Metode Yang Diusulkan	30
3.4.1. Pembagian Dataset	31
3.4.2. Arsitektur 1D-CNN Multi-Branch	34
3.4.3. Strategi Pelatihan Model.....	Error! Bookmark not defined.
3.4.4. Penyetelan Hyperparameter Tuning	36
3.5 Evaluasi Model.....	Error! Bookmark not defined.
3.5.1. Metrik Evaluasi	Error! Bookmark not defined.
3.5.2. Prosedur Evaluasi.....	Error! Bookmark not defined.
3.6 Integrasi Edge AI dan Pengujian Akhir.....	38
3.6.1. Konversi Model ONNX	38
3.6.2. Perancangan Aplikasi Android.....	39
3.6.3. Skenario Pengujian Lapangan.....	43
BAB IV HASIL PENELITIAN DAN PEMBAHASAN	44
4.1 Implementasi Aplikasi Akuisisi Data	44
4.2 Pengumpulan dan Pembentukan <i>Dataset</i>	45
4.2.1. Akuisisi Data Lapangan	45
4.2.2. Pelabelan Data.....	46
4.2.3. Pra-Pemrosesan Dataset	48
4.3 Pelatihan Model (Model Training).....	50

4.3.1.	Pembagian Dataset (<i>Trip-Based Split</i>)	51
4.3.2.	Normalisasi Dataset	52
4.3.3.	Pelatihan Model	53
4.3.4.	Optimasi Hyperparameter (Hyperparameter Tuning)	54
4.3.5.	Hasil Pelatihan Model	55
4.4	Evaluasi Model.....	56
4.4.1.	Hasil Evaluasi pada Holdout Test Set	56
4.4.2.	Analisis Confusion Matrix	58
4.4.3.	Analisis Precision–Recall Curve.....	60
4.4.4.	Analisis Kinerja Model terhadap Setiap Kelas Error! Bookmark not defined.	
4.5	Integrasi Edge AI (Deployment)	60
4.5.1.	Konversi Model PyTorch ke Format ONNX	61
4.5.2.	Implementasi ONNX Runtime pada Android	61
4.5.3.	Pengujian Fungsional Sistem	63
4.6	Pengujian Lapangan dan Pembahasan Implementasi (<i>Field Testing</i>)	64
4.6.1.	Hasil Pengujian Lapangan	64
4.6.2.	Pembahasan Implementasi Sistem	64
BAB V	KESIMPULAN DAN PENELITIAN SELANJUTNYA	66
5.1	Kesimpulan	66
5.2	Saran.....	67
DAFTAR PUSTAKA		69

DAFTAR GAMBAR

Gambar III.1 Diagram Alur Penelitian.....	23
Gambar 3.III.2 Posisi Smartphone	27
Gambar 3.3 Diagram Alur Pemodelan	30
Gambar III.4 Arsitektur Lightweight InceptionTime 1D-CNN	34
Gambar III.5 Detail Struktur Inception Block	34
Gambar III.6 Diagram Use Case Aplikasi Deteksi Anomali Jalan	41
Gambar III.7 Diagram Flowchart Alur Akuisisi dan Inferensi	42
Gambar III.8 Diagram ERD Aplikasi.....	43
Gambar IV.1 Contoh Hasil Data CSV.....	45
Gambar IV.2 Lintasan yang Ditempuh di Kabupaten Pemalang.	46
Gambar IV.3 Distribusi Data Hasil Pelabelan.....	47
Gambar IV.4 Resampling pada Kelas Pothole	48
Gambar IV.5 Resampling pada Kelas Speed Bump.....	49
Gambar IV.6 Distribusi Kelas Dataset	50
Gambar IV.7 Hasil Pembagian Dataset menggunakan Metode Trip-Based Split.	51
Gambar IV.8 Hasil Global Z-Score Normalization.....	52
Gambar IV.9 Kurva Train Loss, Val Loss dan Val Macro F1	55
Gambar IV.10 Hasil Evaluasi Model Pada Holdout Test Set.....	56
Gambar IV.11 Perbandingan Sinyal Pothole (TP vs FN)	57
Gambar IV.12 Confusion Matrix hasil evaluasi model pada Holdout Test Set.....	59
Gambar IV.13 Kurva Precision–Recall untuk masing-masing kelas.	60
Gambar IV.14 Pipeline Inferensi Model ONNX.....	62

DAFTAR LAMPIRAN

Berisi daftar lampiran

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kondisi permukaan jalan merupakan salah satu aspek penting dalam menjaga keselamatan berkendara, kelancaran mobilitas masyarakat, dan efektivitas distribusi logistik. Oleh karena itu, pemantauan kondisi jalan secara berkala menjadi bagian penting dalam mendukung pemeliharaan infrastruktur agar kerusakan dapat diidentifikasi dan ditangani sejak dini. Seiring berkembangnya konsep *Smart City* dan *Internet of Things (IoT)*, sistem pemantauan infrastruktur mulai bertransformasi menjadi lebih cerdas dan berbasis data. Salah satu penerapannya adalah pemantauan kondisi permukaan jalan, di mana data dari perangkat bergerak dimanfaatkan untuk mengidentifikasi kerusakan jalan secara otomatis (Safyari et al., 2024).

Salah satu pendekatan yang banyak dikembangkan adalah pemanfaatan sensor bawaan pada smartphone. Perangkat tersebut dilengkapi sensor akselerometer, giroskop, dan GPS yang mampu merekam dinamika kendaraan selama berkendara. Data yang dihasilkan berupa *multivariate time-series* yang merepresentasikan getaran kendaraan ketika melintasi berbagai kondisi permukaan jalan, sehingga anomali seperti *pothole* dan *speed bump* dapat dikenali melalui analisis sinyal (Grouios et al., 2023). Dalam konteks Kabupaten Pematang Jaya yang memiliki cakupan jaringan jalan kabupaten dan kecamatan yang cukup luas, proses pemantauan kondisi jalan secara manual memerlukan waktu dan sumber daya yang besar. Oleh karena itu, pendekatan berbasis smartphone berpotensi menjadi alternatif pemantauan yang lebih efisien melalui partisipasi pengguna jalan dalam proses pengumpulan data (Utama et al., 2023).

Pemanfaatan sensor smartphone masih menghadapi tantangan. Data yang dihasilkan dipengaruhi oleh dinamika kendaraan, kondisi permukaan jalan, dan variasi kecepatan berkendara sehingga karakteristik sinyal tidak selalu konsisten (Zhao et al., 2024). Kondisi ini meningkatkan kompleksitas klasifikasi karena batas antara sinyal kerusakan jalan dan jalan normal menjadi semakin sulit dibedakan.

Oleh karena itu, diperlukan metode prapemrosesan untuk menghasilkan representasi data yang lebih konsisten sebelum klasifikasi dilakukan (Koukaras & Tjortjis, 2025).

Penelitian terdahulu banyak menggunakan algoritma *Machine Learning* klasik seperti SVM, KNN, dan Random Forest berdasarkan fitur statistik hasil *feature engineering* (Vergni & Todisco, 2023). Pendekatan ini mampu memberikan performa yang baik, tetapi sangat bergantung pada fitur yang dirancang manual sehingga belum sepenuhnya mampu merepresentasikan hubungan temporal yang kompleks pada data *multivariate time-series*. Akibatnya, diperlukan pendekatan yang mampu mempelajari representasi fitur secara otomatis langsung dari data mentah (Ferrari et al., 2021).

Perkembangan *Deep Learning* memberikan pendekatan baru melalui kemampuan *end-to-end learning*. Salah satu arsitektur yang banyak digunakan adalah *One-Dimensional Convolutional Neural Network* (1D-CNN), yang mampu mengekstraksi pola temporal pada sinyal tanpa bergantung pada *feature engineering* manual (Ozoglu & Gökgöz, 2023). Arsitektur ini dipilih karena mampu untuk meningkatkan deteksi kerusakan jalan berbasis data sensor smartphone. Meskipun demikian, aspek evaluasi pada penelitian terdahulu masih memiliki keterbatasan karena sebagian penelitian menggunakan *random split* yang berpotensi menghasilkan estimasi performa terlalu optimistis. Selain itu, model yang semakin kompleks juga menimbulkan kendala memori, waktu inferensi, dan konsumsi energi ketika diterapkan pada perangkat bergerak (Heydari & Mahmoud, 2025; Kim & Kim, 2024).

Berdasarkan latar belakang tersebut, penelitian ini berfokus pada sistem deteksi kerusakan jalan berbasis *Edge AI* menggunakan model *Lightweight 1D-CNN* yang diimplementasikan secara langsung pada perangkat Android. Sistem dirancang untuk mengintegrasikan tahapan prapemrosesan data sensor, strategi evaluasi berbasis lintasan perjalanan (*Trip-Based Holdout Split*), dan *on-device inference* dalam satu alur pemrosesan. Dengan pendekatan tersebut, sistem yang dikembangkan diharapkan mampu memberikan keseimbangan antara akurasi

deteksi, efisiensi komputasi, dan kesiapan implementasi untuk mendukung pemantauan kondisi jalan secara berkelanjutan (Heydari & Mahmoud, 2025).

1.2 Rumusan Masalah

Berdasarkan latar belakang yang sudah dijabarkan, rumusan masalah penelitian ini dapat dirumuskan sebagai berikut:

- 1 Bagaimana mengembangkan sistem deteksi kerusakan jalan berbasis *Edge AI* yang mampu melakukan akuisisi dan pemrosesan data sensor smartphone secara *real-time* pada perangkat Android?
- 2 Bagaimana membangun model *Lightweight One-Dimensional Convolutional Neural Network* (1D-CNN) yang mampu mendeteksi anomali jalan serta diimplementasikan secara efisien melalui *on-device inference* pada perangkat Android?
- 3 Bagaimana mengevaluasi kemampuan generalisasi model menggunakan strategi *Trip-Based Holdout Split* sehingga hasil pengujian lebih merepresentasikan kondisi operasional di lapangan?

1.3 Batasan Masalah

Batasan penelitian pada tugas akhir ini adalah sebagai berikut.

1. Penelitian difokuskan pada deteksi anomali permukaan jalan beraspal di wilayah Kabupaten Pematang Jaya.
2. Objek deteksi dibatasi pada tiga kelas, yaitu Pothole (lubang jalan), Speed Bump (polisi tidur), dan Non-Event (kondisi jalan normal). Sistem tidak mengevaluasi tingkat keparahan kerusakan jalan, melainkan hanya melakukan klasifikasi jenis anomali.
3. Data yang digunakan berasal dari sensor bawaan smartphone, yaitu akselerometer, giroskop, dan GPS.
4. Smartphone dipasang pada holder stang kendaraan dengan orientasi tegak (portrait) menghadap ke depan selama proses pengambilan data dan pengujian.
5. Sistem diuji pada kondisi kendaraan dengan kecepatan minimal 15 km/jam.

6. Pengambilan data dan pengujian dilakukan menggunakan satu kendaraan roda dua tipe skuter matik dengan sistem suspensi standar bawaan pabrik.
7. Evaluasi performa model dilakukan menggunakan strategi Trip-Based Holdout Split berdasarkan lintasan perjalanan.
8. Sistem dirancang untuk melakukan proses inferensi secara langsung pada perangkat Android (*on-device inference*) tanpa melibatkan pemrosesan berbasis *cloud*.

1.4 Tujuan Penelitian

Tujuan penelitian pada tugas akhir ini adalah sebagai berikut:

1. Mengembangkan sistem deteksi kerusakan jalan berbasis *Edge AI* yang mampu melakukan akuisisi, pemrosesan, dan deteksi data sensor smartphone secara *real-time* pada perangkat Android.
2. Membangun model *Lightweight One-Dimensional Convolutional Neural Network* (1D-CNN) serta mengimplementasikannya melalui mekanisme *on-device inference* pada perangkat Android untuk mendeteksi anomali jalan secara *real-time*.
3. Mengevaluasi kemampuan generalisasi model menggunakan strategi *Trip-Based Holdout Split* sehingga diperoleh pengukuran performa yang lebih merepresentasikan kondisi operasional di lapangan.

1.5 Manfaat Penelitian

1.5.1. Manfaat Akademis

Manfaat akademis yang diharapkan pada penelitian ini yaitu:

- a. Memberikan referensi ilmiah mengenai penerapan model *Lightweight One-Dimensional Convolutional Neural Network* (1D-CNN) untuk mendeteksi anomali permukaan jalan berdasarkan data *multivariate time-series* yang diperoleh dari sensor smartphone.
- b. Memberikan alternatif strategi evaluasi model menggunakan *Trip-Based Holdout Split* sehingga dapat menjadi rujukan dalam pengembangan penelitian deteksi kerusakan jalan yang menekankan

kemampuan generalisasi model pada kondisi operasional di lapangan.

- c. Menjadi referensi bagi penelitian selanjutnya mengenai implementasi *Edge AI* untuk pemrosesan data sensor secara langsung pada perangkat bergerak (*on-device inference*) tanpa bergantung pada layanan *cloud*.

1.5.2. Manfaat Praktis

Manfaat praktis yang diharapkan pada penelitian ini yaitu:

- a. Memberikan alternatif sistem deteksi kerusakan jalan berbasis smartphone yang dapat dimanfaatkan sebagai pendukung proses identifikasi awal kondisi jalan sehingga membantu Dinas Pekerjaan Umum Kabupaten Pematang Jaya dalam menentukan prioritas pemeliharaan jalan secara lebih efisien.
- b. Menghasilkan informasi lokasi anomali permukaan jalan yang dapat dimanfaatkan sebagai pendukung proses pemantauan kondisi jalan serta menjadi dasar pengembangan layanan transportasi cerdas pada penelitian selanjutnya.

BAB II

LANDASAN TEORI

2.1 Tinjauan Studi

Terdapat beberapa penelitian terkait yang dilakukan oleh peneliti sebelumnya dan digunakan pada penelitian ini sebagai landasan, referensi, atau acuan oleh penulis dalam mendukung penelitian tentang klasifikasi anomali jalan menggunakan sensor inersia smartphone dan algoritma pembelajaran mendalam. Dari referensi yang diambil, terdapat persamaan pada objek getaran sensor inersia dan metode klasifikasi berbasis *Deep Learning*.

1. (Budi Darma et al., 2022) melakukan penelitian mengenai segmentasi semantik pada data sensor gerak smartphone untuk pemantauan permukaan jalan. Penelitian tersebut mengusulkan arsitektur U-Net yang dimodifikasi dengan Bidirectional LSTM (BiLSTM) untuk mengelompokkan segmen sinyal sensor ke dalam beberapa kelas kondisi jalan. Hasil eksperimen menunjukkan bahwa penggunaan fitur z-axis accelerometer dan z-axis gyroscope memberikan performa terbaik, serta model yang diusulkan mampu mengungguli beberapa metode segmentasi semantik yang telah ada. Berdasarkan hasil tersebut, penelitian ini menunjukkan bahwa integrasi U-Net dan BiLSTM efektif untuk segmentasi data sensor smartphone pada klasifikasi kondisi permukaan jalan.
2. (Ozoglu & Gökgöz, 2023) melakukan penelitian mengenai deteksi lubang jalan (*pothole*) menggunakan data getaran yang diperoleh dari sensor akselerometer dan giroskop pada smartphone. Data sensor ditransformasikan ke dalam bentuk representasi citra (*pixel-based images*) sebelum dimasukkan ke beberapa konfigurasi arsitektur CNN. Hasil penelitian menunjukkan bahwa model yang diusulkan mencapai akurasi validasi sebesar **93,24%** dan performa deteksi lapangan sekitar **80–87%**. Berdasarkan hasil tersebut, penelitian ini menunjukkan bahwa pendekatan CNN berpotensi digunakan untuk deteksi lubang jalan berbasis sensor smartphone.

3. (Kim & Kim, 2024) melakukan penelitian mengenai deteksi kerusakan jalan pada perangkat bergerak menggunakan model *Lightweight Deep Learning*. Penelitian tersebut mengembangkan mekanisme pengumpulan data otomatis dan model CNN untuk mengklasifikasikan *speed bump*, *manhole*, dan *pothole* dari data sensor smartphone. Hasil penelitian menunjukkan bahwa model yang diusulkan memiliki performa yang baik serta lebih efisien dari sisi kecepatan pemrosesan. Berdasarkan hasil tersebut, penelitian ini menunjukkan bahwa pendekatan CNN ringan dapat diterapkan secara praktis pada smartphone untuk deteksi kerusakan jalan.
4. (Arce-saenz et al., 2025) melakukan penelitian mengenai klasifikasi kondisi permukaan jalan menggunakan data Multi-Inertial Measurement Unit (Multi-IMU). Penelitian tersebut membandingkan beberapa arsitektur Deep Learning, yaitu Single-head CNN, Multi-head CNN, Single-head CNN-LSTM, dan Multi-head CNN-LSTM. Hasil penelitian menunjukkan bahwa model Single-head CNN-LSTM memberikan performa terbaik dengan Macro F1-score sebesar 0,9338. Berdasarkan hasil tersebut, penelitian ini menunjukkan bahwa arsitektur hibrida CNN-LSTM mampu meningkatkan kemampuan klasifikasi kondisi permukaan jalan berbasis data sensor inersia.
5. (Zareei et al., 2025) melakukan penelitian mengenai deteksi anomali permukaan jalan berbasis sensor smartphone untuk mendukung sistem *mobile crowd-sensing*. Penelitian tersebut mengembangkan sistem yang memanfaatkan data akselerometer, giroskop, dan GPS yang diproses melalui tahapan prapemrosesan, termasuk *moving average filtering* dan *sensor fusion*, kemudian diklasifikasikan menggunakan arsitektur *2-CNN*. Hasil penelitian menunjukkan bahwa model yang diusulkan mampu mencapai akurasi sebesar **98%** dalam membedakan *pothole* dan *speed bump*. Berdasarkan hasil tersebut, penelitian ini menunjukkan bahwa pendekatan CNN berbasis sensor smartphone memiliki potensi untuk diterapkan sebagai sistem pemantauan kondisi jalan.

6. (Aguilar-gonzález & Medina Santiago, 2025) melakukan penelitian mengenai deteksi peristiwa jalan menggunakan sinyal getaran dan akselerasi multiaxial. Penelitian tersebut mengembangkan pendekatan berbasis CNN untuk mengklasifikasikan beberapa kategori peristiwa jalan, termasuk *speed bump*, *pothole*, dan *sudden braking*. Hasil penelitian menunjukkan bahwa model yang diusulkan mencapai akurasi 93,51% dengan nilai AUC 0,9855. Berdasarkan hasil tersebut, penelitian ini menunjukkan bahwa CNN dapat digunakan secara efektif untuk deteksi peristiwa jalan berbasis sinyal sensor pada lingkungan yang dinamis.
7. (Aguilar-gonzález & Medina Santiago, 2026) melakukan penelitian mengenai deteksi peristiwa jalan yang sadar konteks (*context-aware*) menggunakan arsitektur hibrida CNN-BiLSTM. Penelitian tersebut memanfaatkan data akselerasi dan getaran multiaxial untuk menangkap fitur jangka pendek sekaligus ketergantungan temporal yang lebih panjang. Hasil penelitian menunjukkan bahwa model yang diusulkan mencapai akurasi global **95,91%** dan *macro F1-score* **0,959**. Berdasarkan hasil tersebut, penelitian ini menunjukkan bahwa kombinasi CNN dan BiLSTM efektif untuk membedakan peristiwa jalan yang memiliki pola sinyal yang mirip.

Tabel II.1 *State of the art*

No	Judul Artikel	Masalah	Metode	Hasil
1	Semantic segmentation on smartphone motion sensor data for road surface monitoring (Budi Darma et al., 2022)	Klasifikasi sinyal tradisional (berbasis window tetap) seringkali gagal melabeli rentetan waktu secara presisi ketika pola	Penggunaan pendekatan Segmentasi Semantik (1D U-Net dan BiLSTM) untuk melabeli anomali secara titik-per-titik (point-wise) dari	Arsitektur Deep Learning (U-Net BiLSTM) mengungguli pendekatan klasikal dalam melokalisasi 6 kelas kondisi aspal, termasuk

No	Judul Artikel	Masalah	Metode	Hasil
		guncangan sepeda motor saling tumpang tindih.	sinyal akselerometer sepeda motor.	lubang dan polisi tidur.
2	Detection of Road Potholes by Applying Convolutional Neural Network Method Based on Road Vibration Data (Ozoglu & Gökgez, 2023)	Kesulitan mendeteksi lubang jalan (potholes) secara otomatis menggunakan getaran tanpa terpengaruh oleh noise berkendara.	Transformasi sinyal getaran akselerometer dan giroskop menjadi representasi citra (<i>pixel-based image</i>), kemudian diklasifikasikan menggunakan beberapa konfigurasi arsitektur CNN.	Menghasilkan akurasi tinggi dalam mengklasifikasi lubang jalan dari data getaran inersia.
3	A Road Defect Detection System Using Smartphones (Kim & Kim, 2024)	Deteksi kerusakan jalan secara real-time menggunakan smartphone dengan keterbatasan daya dan komputasi lokal.	Algoritma CNN yang melatih data runtun getaran akselerometer smartphone untuk klasifikasi cacat jalan.	Sistem deteksi cacat jalan terintegrasi smartphone yang hemat sumber daya dengan performa klasifikasi yang andal.

No	Judul Artikel	Masalah	Metode	Hasil
4	Exploring single-head and multi-head CNN and LSTM-based models for road surface classification (Arce-saenz et al., 2025)	Penurunan akurasi pemetaan anomali bila hanya menggunakan Single-IMU dan kurangnya memori sekuensial pada CNN murni.	Membandingkan topologi Single-head vs Multi-head CNN, serta arsitektur hibrida CNN+LSTM berbasis data Multi-IMU.	Arsitektur CNN+LSTM unggul mencetak macro F1-score 0.9338 dibanding model CNN biasa dalam klasifikasi permukaan.
5	Machine learning model for road anomaly detection using smartphone accelerometer data (Zareei et al., 2025)	Kebutuhan sistem monitoring kondisi jalan yang sangat ringan dan dapat diterapkan skala masif (mobile crowd-sensing).	Moving average filtering dipadukan dengan arsitektur 2-CNN untuk memproses 6 fitur sensor dasar secara efisien.	Mencapai akurasi 98% untuk klasifikasi lubang dan polisi tidur dengan beban komputasi minimal.
6	CNN-Based Road Event Detection Using Multiaxial Vibration and Acceleration Signals (Aguilar-	Metode ekstraksi fitur klasikal (non-Deep Learning) rentan gagal di lingkungan dinamis yang	Jaringan CNN murni yang memecah input berbasis temporal windows untuk menangani akselerasi multi-	Mengalahkan metode rule-based sebesar 10% dan meraih akurasi 93.5% dengan efisiensi tinggi pada

No	Judul Artikel	Masalah	Metode	Hasil
	gonzález & Medina Santiago, 2025)	bergetar secara frekuensi tinggi.	aksial secara instan.	embedded hardware.
7	Context-Aware Road Event Detection Using Hybrid CNN-BiLSTM Networks (Aguilar-gonzález & Medina Santiago, 2026)	CNN biasa gagal membedakan pola guncangan yang berdurasi panjang (long-range dependencies) akibat keterbatasan ukuran kernel (receptive field).	Ekstraksi fitur gelombang instan oleh 1D-CNN dikawinkan dengan pemodelan urutan memori berarah ganda oleh gerbang BiLSTM.	Sangat andal membedakan peristiwa bertekstur mirip (akurasi 95.91%) berkat pemahaman konteks spatio-temporal komprehensif.

Berdasarkan hasil sintesis penelitian terdahulu, dapat disimpulkan bahwa pengembangan sistem deteksi kerusakan jalan berbasis sensor smartphone telah mengalami kemajuan yang signifikan, terutama melalui penerapan berbagai arsitektur *Deep Learning*. Namun, sebagian besar penelitian masih berfokus pada peningkatan performa model klasifikasi atau pengembangan arsitektur tertentu secara terpisah. Integrasi antara tahapan prapemrosesan data sensor, strategi evaluasi yang mampu merepresentasikan kemampuan generalisasi model secara lebih objektif, serta implementasi *on-device inference* pada perangkat Android masih relatif terbatas. Selain itu, penggunaan strategi pembagian data secara acak pada beberapa penelitian berpotensi menghasilkan estimasi performa yang terlalu optimistis karena belum sepenuhnya mengisolasi data berdasarkan lintasan perjalanan. Oleh karena itu, penelitian ini mengembangkan sistem deteksi kerusakan jalan berbasis *Edge AI* menggunakan *Lightweight One-Dimensional Convolutional Neural Network* (1D-CNN) yang mengintegrasikan tahapan

prapemrosesan data sensor, evaluasi berbasis *Trip-Based Holdout Split*, dan *on-device inference* pada perangkat Android dalam satu alur pemrosesan. Pendekatan tersebut diharapkan mampu menghasilkan sistem yang tidak hanya memiliki performa deteksi yang baik, tetapi juga efisien dan lebih representatif terhadap kondisi operasional di lapangan.

2.2 Tinjauan Pustaka

2.2.1. Anomali Permukaan Jalan Beraspal

Kerusakan dan rintangan pada permukaan jalan beraspal merupakan salah satu faktor yang memengaruhi kenyamanan berkendara, kestabilan kendaraan, umur pakai komponen suspensi, serta keselamatan lalu lintas. Kondisi permukaan jalan yang tidak rata menyebabkan perubahan gaya yang diterima roda kendaraan sehingga menghasilkan respons dinamis berupa perubahan percepatan dan orientasi kendaraan. Respons tersebut dapat direkam menggunakan sensor inersia dan dimanfaatkan sebagai dasar dalam proses deteksi anomali permukaan jalan (Safyari et al., 2024). Pada penelitian ini, objek anomali dibatasi pada dua jenis utama, yaitu sebagai berikut.

1. Lubang Jalan (*Pothole*)

Pothole merupakan salah satu bentuk kerusakan yang ditandai dengan terbentuknya cekungan pada permukaan jalan akibat degradasi lapisan perkerasan. Kerusakan ini umumnya berkembang melalui proses yang melibatkan infiltrasi air, retak akibat kelelahan (*fatigue cracking*), pelepasan material perkerasan, serta beban lalu lintas berulang (Safyari et al., 2024). Ketika kendaraan melintasi *pothole*, terjadi perubahan percepatan yang bersifat mendadak akibat perbedaan elevasi permukaan jalan. Karakteristik tersebut menghasilkan pola respons dinamis yang berbeda dibandingkan kondisi jalan normal sehingga *pothole* menjadi salah satu anomali yang paling banyak digunakan sebagai objek penelitian pada sistem deteksi kerusakan jalan berbasis sensor.

2. Polisi Tidur (*Speed Bump*)

Speed bump merupakan fasilitas pengendali lalu lintas berupa tonjolan melintang yang dirancang untuk memaksa kendaraan mengurangi kecepatan pada lokasi tertentu. Berbeda dengan *pothole* yang merupakan bentuk kerusakan jalan, *speed bump* merupakan elemen infrastruktur yang sengaja dibangun dengan dimensi dan bentuk geometris tertentu. Ketika kendaraan melewati *speed bump*, suspensi kendaraan menghasilkan respons percepatan yang relatif lebih teratur karena kendaraan mengalami fase menaiki dan menuruni tonjolan secara berurutan. Meskipun bukan merupakan kerusakan jalan, karakteristik respons tersebut tetap menghasilkan pola sinyal yang berbeda dari kondisi jalan normal sehingga dalam penelitian berbasis sensor smartphone, *speed bump* sering diperlakukan sebagai kelas anomali tersendiri (Zareei et al., 2025).

2.2.2. Sensor Smartphone (IMU dan GPS)

Smartphone modern telah dilengkapi berbagai sensor berbasis *Micro-Electro-Mechanical Systems* (MEMS) yang mampu merekam informasi mengenai gerakan, orientasi, dan posisi perangkat secara real-time. Perkembangan teknologi MEMS menjadikan smartphone sebagai platform akuisisi data yang efisien karena mampu mengintegrasikan beberapa sensor dalam satu perangkat berukuran kecil dengan konsumsi daya yang rendah. Dalam penelitian deteksi anomali permukaan jalan, kombinasi sensor *Inertial Measurement Unit* (IMU) dan *Global Positioning System* (GPS) banyak dimanfaatkan untuk memperoleh informasi mengenai respons dinamis kendaraan serta lokasi terjadinya anomali (Capuano et al., 2023).

1. Akselerometer (Accelerometer)

Akselerometer merupakan sensor yang mengukur percepatan linier pada tiga sumbu ortogonal (x , y , dan z). Sensor ini merekam perubahan percepatan yang dialami perangkat akibat pergerakan kendaraan, termasuk pengaruh gravitasi dan gaya inersia yang timbul selama proses berkendara. Dalam sistem deteksi anomali jalan, data akselerometer digunakan untuk merepresentasikan karakteristik getaran kendaraan ketika melintasi

berbagai kondisi permukaan jalan sehingga menjadi sumber informasi utama dalam proses klasifikasi (Capuano et al., 2023).

2. **Giroskop (Gyroscope)**

Giroskop merupakan sensor yang mengukur kecepatan sudut (*angular velocity*) terhadap tiga sumbu rotasi. Berbeda dengan akselerometer yang merekam perubahan percepatan linier, giroskop memberikan informasi mengenai perubahan orientasi atau rotasi perangkat selama kendaraan bergerak. Kombinasi data akselerometer dan giroskop memungkinkan representasi gerakan kendaraan menjadi lebih komprehensif sehingga perubahan orientasi akibat kontur permukaan jalan dapat dianalisis dengan lebih baik (Capuano et al., 2023).

3. **Global Positioning System (GPS)**

Global Positioning System (GPS) merupakan sistem navigasi berbasis satelit yang digunakan untuk menentukan posisi geografis perangkat berdasarkan koordinat lintang (*latitude*) dan bujur (*longitude*). Selain informasi lokasi, GPS juga dapat menyediakan data kecepatan dan waktu yang berguna dalam analisis perjalanan. Pada penelitian deteksi kerusakan jalan, data GPS berfungsi untuk mengaitkan hasil klasifikasi dengan lokasi anomali sehingga memungkinkan proses pemetaan kondisi jalan pada wilayah pengamatan (Capuano et al., 2023).

2.2.3. **Deep Learning**

Deep Learning merupakan salah satu cabang *Machine Learning* yang memanfaatkan jaringan saraf tiruan (*Artificial Neural Network*) dengan banyak lapisan (*deep neural network*) untuk mempelajari representasi data secara bertingkat. Berbeda dengan pendekatan *Machine Learning* konvensional yang umumnya bergantung pada proses *feature engineering* secara manual, *Deep Learning* mampu mengekstraksi representasi fitur secara otomatis langsung dari data masukan melalui proses pembelajaran. Jaringan saraf tiruan tersusun atas *input layer*, *hidden layer*, dan *output layer* yang saling terhubung melalui bobot (*weights*). Selama proses pelatihan, bobot tersebut diperbarui secara iteratif

menggunakan mekanisme *forward propagation* dan *backpropagation* untuk meminimalkan kesalahan prediksi berdasarkan fungsi kerugian (*loss function*) (Safyari et al., 2024).

Kemampuan *Deep Learning* dalam mempelajari hubungan yang kompleks dan nonlinier menjadikannya banyak diterapkan pada berbagai jenis data, termasuk citra, suara, bahasa alami, dan data deret waktu (*time-series*). Berbagai arsitektur telah dikembangkan untuk menyesuaikan karakteristik data yang diproses, seperti *Convolutional Neural Network* (CNN), *Recurrent Neural Network* (RNN), dan *Transformer*. Pada penelitian ini, arsitektur yang digunakan adalah *One-Dimensional Convolutional Neural Network* (1D-CNN) karena mampu mengekstraksi pola lokal pada data *multivariate time-series* secara efektif tanpa memerlukan proses *feature engineering* secara manual (Kim & Kim, 2024). Pembahasan mengenai prinsip kerja arsitektur 1D-CNN dijelaskan pada subbab berikutnya.

2.2.4. *One-Dimensional Convolutional Neural Network* (1D-CNN)

One-Dimensional Convolutional Neural Network (1D-CNN) merupakan salah satu arsitektur *Deep Learning* yang dirancang untuk mengolah data satu dimensi, seperti data deret waktu (*time-series*) dan sinyal sensor. Berbeda dengan *Convolutional Neural Network* (CNN) dua dimensi yang mengolah data citra, 1D-CNN menerapkan operasi konvolusi sepanjang dimensi temporal untuk mengekstraksi pola lokal dari urutan data. Kemampuan tersebut menjadikan 1D-CNN banyak digunakan pada berbagai aplikasi yang melibatkan data sekuensial, seperti pengenalan suara, analisis sinyal biomedis, deteksi aktivitas manusia, serta klasifikasi data sensor (Kiranyaz et al., 2021).

Arsitektur 1D-CNN tersusun atas beberapa lapisan (*layer*) yang bekerja secara bertahap untuk mengekstraksi fitur dan menghasilkan prediksi. Lapisan-lapisan utama pada arsitektur 1D-CNN meliputi sebagai berikut.

1. *Convolution Layer*

Convolution layer merupakan lapisan utama yang berfungsi mengekstraksi karakteristik lokal dari data masukan menggunakan

sejumlah *kernel* atau *filter*. Proses konvolusi menghasilkan *feature map* yang merepresentasikan pola-pola penting pada sinyal sehingga dapat digunakan pada proses pembelajaran berikutnya (Kiranyaz et al., 2021).

2. **Activation Layer**

Activation layer berfungsi memperkenalkan sifat nonlinier ke dalam jaringan saraf sehingga model mampu mempelajari hubungan yang lebih kompleks. Salah satu fungsi aktivasi yang umum digunakan adalah *Rectified Linear Unit* (ReLU) karena memiliki proses komputasi yang sederhana dan mampu mempercepat konvergensi selama pelatihan (Kiranyaz et al., 2021).

3. **Pooling Layer**

Pooling layer digunakan untuk mereduksi dimensi *feature map* dengan tetap mempertahankan informasi yang paling representatif. Selain mengurangi kompleksitas komputasi, proses ini juga membantu meningkatkan kemampuan generalisasi model dan mengurangi risiko *overfitting* (Kiranyaz et al., 2021).

4. **Fully Connected Layer**

Fully connected layer berfungsi mengintegrasikan fitur-fitur yang telah diekstraksi pada lapisan sebelumnya menjadi representasi akhir yang digunakan dalam proses klasifikasi (Kiranyaz et al., 2021).

5. **Output Layer**

Output layer merupakan lapisan terakhir yang menghasilkan prediksi kelas berdasarkan hasil pembelajaran model. Pada permasalahan klasifikasi multikelas, lapisan ini umumnya menggunakan fungsi aktivasi *Softmax* untuk menghasilkan probabilitas pada setiap kelas (Kiranyaz et al., 2021).

Kemampuan 1D-CNN dalam mengekstraksi fitur secara otomatis langsung dari data mentah (*end-to-end learning*) menjadikannya sesuai untuk pengolahan data *multivariate time-series* tanpa bergantung pada proses *feature engineering* secara manual. Selain itu, kompleksitas komputasi yang relatif lebih rendah dibandingkan beberapa arsitektur *Deep Learning* lainnya menjadikan 1D-CNN lebih efisien untuk diimplementasikan pada perangkat

dengan sumber daya terbatas. Oleh karena itu, arsitektur ini banyak dimanfaatkan dalam sistem deteksi berbasis sensor, termasuk deteksi anomali permukaan jalan menggunakan smartphone (Kiranyaz et al., 2021).

2.2.5. Edge Computing dan ONNX Runtime

Seiring meningkatnya kompleksitas model *Deep Learning*, proses inferensi tidak lagi selalu dilakukan pada server atau *cloud*. Salah satu pendekatan yang banyak dikembangkan adalah Edge Computing, yaitu paradigma komputasi yang memindahkan proses pemrosesan data lebih dekat dengan sumber data atau perangkat pengguna. Dalam konteks kecerdasan buatan, pendekatan ini dikenal sebagai Edge AI, yaitu implementasi model kecerdasan buatan secara langsung pada perangkat (*on-device inference*) tanpa bergantung pada komputasi berbasis *cloud*. Pendekatan tersebut mampu mengurangi latensi, menghemat penggunaan bandwidth, meningkatkan ketersediaan sistem, serta menjaga privasi data karena proses inferensi dilakukan secara lokal pada perangkat (Li et al., 2022).

Implementasi *Edge AI* memerlukan *inference engine* yang mampu menjalankan model secara efisien pada perangkat dengan sumber daya komputasi terbatas. Salah satu *framework* yang banyak digunakan adalah *Open Neural Network Exchange Runtime* (ONNX Runtime). ONNX Runtime merupakan mesin inferensi lintas platform yang mendukung eksekusi model *Machine Learning* dan *Deep Learning* dalam format ONNX dengan optimasi terhadap penggunaan memori dan kecepatan inferensi. Dukungan terhadap berbagai sistem operasi dan perangkat keras menjadikan ONNX Runtime banyak dimanfaatkan dalam pengembangan aplikasi berbasis *Edge AI*, termasuk implementasi model pada perangkat bergerak (Li et al., 2022).

2.2.6. Segmentasi Data Runtun Waktu (*Time-Series Windowing*)

Data yang dihasilkan oleh sensor seperti akselerometer dan giroskop merupakan data deret waktu (*time-series*) yang tersusun secara berurutan berdasarkan waktu. Berbeda dengan data tabular yang setiap observasinya

bersifat independen, setiap sampel pada data *time-series* memiliki keterkaitan temporal dengan sampel sebelum maupun sesudahnya. Oleh karena itu, data deret waktu umumnya tidak diproses secara langsung sebagai satu rangkaian yang panjang, melainkan dibagi menjadi sejumlah segmen dengan panjang tertentu (*window*) agar karakteristik lokal pada sinyal dapat dianalisis secara lebih efektif (Ferrari et al., 2021).

Proses pembagian data deret waktu menjadi beberapa segmen dikenal sebagai **time-series windowing** atau **windowing**. Setiap *window* berisi sejumlah sampel yang mewakili suatu interval waktu tertentu dan diperlakukan sebagai satu unit data masukan bagi model pembelajaran mesin maupun *Deep Learning*. Pendekatan ini bertujuan mempertahankan hubungan temporal antar sampel dalam setiap segmen, sekaligus menghasilkan ukuran data yang seragam sehingga memudahkan proses pelatihan dan inferensi model (Ferrari et al., 2021).

Berdasarkan cara pembentukannya, *windowing* umumnya dibedakan menjadi *non-overlapping window* dan *overlapping window*. *Non-overlapping window* membagi data menjadi beberapa segmen tanpa adanya irisan antar *window*, sedangkan *overlapping window* memungkinkan sebagian sampel digunakan kembali pada *window* berikutnya. Penggunaan *overlapping window* dapat meningkatkan jumlah data pelatihan dan menjaga kontinuitas informasi temporal antarsegmen, meskipun menghasilkan beban komputasi yang lebih tinggi dibandingkan pendekatan tanpa *overlap* (Ferrari et al., 2021).

Pada aplikasi berbasis data sensor, termasuk pengenalan aktivitas manusia (*Human Activity Recognition*) maupun deteksi anomali permukaan jalan, *time-series windowing* berperan sebagai tahapan awal yang penting sebelum proses klasifikasi dilakukan (Ferrari et al., 2021). Melalui segmentasi data, karakteristik sinyal pada setiap interval waktu dapat direpresentasikan secara lebih konsisten sehingga mempermudah model dalam mempelajari pola-pola yang berkaitan dengan kejadian tertentu.

2.2.7. Evaluation Metrics

Evaluation metrics adalah ukuran kuantitatif untuk menilai kinerja model klasifikasi/deteksi. Dalam praktik evaluasi, hasil prediksi dibandingkan dengan label ground-truth melalui confusion matrix, kemudian metrik digunakan untuk merangkum confusion matrix menjadi satu nilai sehingga model dapat dibandingkan atau dipilih (Opitz, 2024).

Pada tugas deteksi kejadian seperti kerusakan jalan, pemilihan metrik sebaiknya mempertimbangkan tujuan operasional karena setiap metrik menonjolkan aspek kinerja yang berbeda. Selain itu, pada data yang tidak seimbang, akurasi cenderung sangat dipengaruhi oleh prevalensi kelas sehingga perlu kehati-hatian dalam interpretasi (Opitz, 2024). Pada penelitian ini, metrik utama yang digunakan adalah *Precision*, *Recall*, *F1-Score*, dan *PR-AUC*.

1. Confusion Matrix dan Notasi

Confusion matrix menyajikan tabulasi antara label aktual dan prediksi model. Untuk kasus biner, notasi yang digunakan adalah True Positive (TP), False Positive (FP), True Negative (TN), False Negative (FN). Konsep ini sejalan dengan definisi confusion matrix sebagai matriks yang menghitung massa kejadian untuk setiap kombinasi prediksi–label, yang kemudian diringkas menjadi metrik (Opitz, 2024).

2. Accuracy (Akurasi)

Akurasi mengukur proporsi prediksi benar terhadap seluruh sampel. Namun, akurasi sangat dipengaruhi oleh distribusi/prevalensi kelas pada data uji sehingga pada kondisi tidak seimbang dapat memberi gambaran yang kurang representative (Opitz, 2024).

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \quad (1)$$

3. Precision (Presisi)

Presisi mengukur ketepatan prediksi positif. Secara konsep, precision dapat dipahami sebagai rasio “prediksi kelas *i* yang benar”

dibanding seluruh prediksi kelas i , dan juga dapat diinterpretasikan sebagai pendekatan probabilitas kelas benar ketika model memprediksi kelas tersebut (Opitz, 2024).

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

4. *Recall* (Sensitivitas, True Positive Rate)

Recall mengukur kelengkapan deteksi positif. Secara konsep, recall adalah rasio “prediksi benar pada kelas i ” dibanding seluruh data yang benar-benar termasuk kelas i , dan dapat dipahami sebagai pendekatan probabilitas prediksi benar ketika input memang berasal dari kelas tersebut (Opitz, 2024).

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

5. *F1-Score*

F1 adalah rata-rata harmonik dari precision dan recall sehingga bernilai tinggi hanya jika keduanya sama-sama tinggi. Konsep penggabungan precision–recall ini juga dijelaskan dalam bentuk definisi berbasis mean harmonik pada pembahasan F1 (Opitz, 2024).

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

6. Precision–Recall Curve dan PR-AUC

Precision-Recall Curve (PR Curve) menggambarkan hubungan antara nilai precision dan recall pada berbagai ambang batas (threshold) probabilitas prediksi. Kurva ini lebih akurat dibandingkan kurva ROC untuk mengevaluasi data dengan ketidakseimbangan kelas (imbalanced dataset), seperti kasus anomali permukaan jalan, karena perhitungan PR tidak melibatkan komponen True Negative (TN) dari data kelas normal yang mendominasi (Richardson et al., 2024).

Untuk merangkum performa model ke dalam satu nilai skalar tunggal, dihitunglah luasan area di bawah kurva tersebut atau Area Under the Precision-Recall Curve (PR-AUC) (Richardson et al., 2024). Nilai PR-AUC berkisar dari 0 hingga 1. Karena hasil prediksi model

bersifat diskrit, perhitungan PR-AUC umumnya diestimasi secara numerik menggunakan pendekatan Average Precision (AP) dengan rumus berikut :

$$\text{PR-AUC} \approx \text{AP} = \sum_n (R_n - R_{n-1})P_n \quad (5)$$

P_n dan R_n adalah nilai *precision* dan *recall* pada *threshold* ke- n . Perhitungan ini menjumlahkan nilai *precision* di setiap *threshold* yang dibobotkan dengan peningkatan *recall* dari *threshold* sebelumnya (Richardson et al., 2024).

2.3 Kerangka Pemikiran

Alur logis kerangka pemikiran dalam penelitian ini digambarkan pada diagram berikut

Tabel II.2 Kerangka Pemikiran

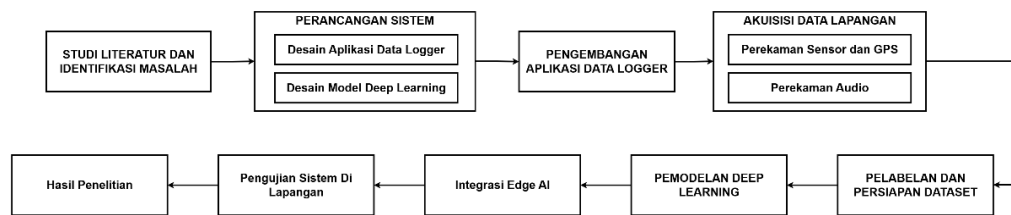
Identifikasi Masalah
Pemantauan kondisi jalan masih didominasi inspeksi manual yang kurang efisien. Pemanfaatan sensor smartphone menghadapi tantangan berupa variasi karakteristik sinyal, kompleksitas klasifikasi data <i>multivariate time-series</i> , keterbatasan evaluasi model, dan implementasi pada perangkat bergerak.
Tujuan
Mengembangkan sistem deteksi anomali permukaan jalan berbasis Edge AI menggunakan Lightweight 1D-CNN yang mampu melakukan deteksi secara <i>real-time</i> pada perangkat Android.
Metode
Penelitian menggunakan data primer yang diperoleh dari sensor akselerometer, giroskop, dan GPS pada smartphone. Data diproses melalui tahapan pra-pemrosesan, segmentasi data <i>time-series</i> , kemudian digunakan untuk melatih model Lightweight 1D-CNN. Evaluasi model dilakukan menggunakan strategi <i>Trip-Based Holdout Split</i> dengan metrik Accuracy, Precision, Recall, dan F1-Score. Model yang telah dilatih selanjutnya diimplementasikan pada aplikasi Android menggunakan ONNX Runtime untuk mendukung mekanisme <i>on-device inference</i> .
Hasil
Menghasilkan sistem deteksi anomali permukaan jalan berbasis Android yang mampu melakukan <i>on-device inference</i> secara <i>real-time</i> dengan performa deteksi yang baik dan efisiensi komputasi yang tinggi.

BAB III

METODE PENELITIAN

3.1 Alur Penelitian

Penelitian ini dilaksanakan melalui serangkaian tahapan yang saling berkesinambungan, dimulai dari tahap perancangan sistem, pengembangan aplikasi akuisisi data, pengumpulan dan persiapan dataset, pembangunan model *Deep Learning*, hingga implementasi model pada perangkat Android berbasis *Edge AI*. Setiap tahapan menghasilkan luaran yang menjadi masukan bagi tahapan berikutnya sehingga membentuk suatu alur penelitian yang sistematis. Alur penelitian yang digunakan pada penelitian ini ditunjukkan pada Gambar 3.1.



Gambar III.1 Diagram Alur Penelitian

Berdasarkan Gambar 3.1, tahapan penelitian yang dilakukan adalah sebagai berikut.

1. Studi Literatur dan Identifikasi Masalah

Tahap awal dilakukan dengan mengidentifikasi permasalahan penelitian melalui studi literatur mengenai deteksi anomali permukaan jalan, pemanfaatan sensor smartphone, *Deep Learning*, serta implementasi *Edge AI*. Hasil kajian ini digunakan sebagai dasar dalam merumuskan tujuan penelitian, kebutuhan sistem, dan metode yang digunakan.

2. Perancangan Sistem

Tahap ini meliputi perancangan arsitektur sistem secara keseluruhan, perancangan aplikasi *data logger*, serta perancangan arsitektur model *Deep Learning* yang akan digunakan untuk mendeteksi anomali permukaan jalan.

3. Pengembangan Aplikasi Data Logger

Aplikasi Android dikembangkan sebagai *data logger* untuk merekam data sensor akselerometer, giroskop, GPS, dan informasi pendukung lainnya selama proses pengambilan data lapangan. Aplikasi ini berfungsi sebagai sarana akuisisi data dan belum melakukan proses inferensi.

4. Akuisisi Data Lapangan

Data primer dikumpulkan melalui survei lapangan menggunakan sepeda motor dengan smartphone yang dipasang pada *holder*. Selama proses perekaman dilakukan pencatatan data sensor, koordinat GPS, serta rekaman suara yang digunakan sebagai referensi dalam proses pelabelan data.

5. Pelabelan dan Persiapan Dataset

Data hasil perekaman diproses melalui tahapan *peak detection* untuk memperoleh kandidat kejadian (*event*), dilanjutkan dengan pelabelan manual berdasarkan referensi rekaman suara. Dataset kemudian dipersiapkan melalui proses *resampling*, pembentukan *event-centered window*, pembagian data berdasarkan *trip*, serta normalisasi Z-Score sebelum digunakan pada proses pelatihan model.

6. Pemodelan Deep Learning

Dataset yang telah dipersiapkan digunakan untuk melatih model Lightweight InceptionTime1D-CNN. Tahap ini meliputi proses pelatihan model, augmentasi data, penyetelan hiperparameter menggunakan Optuna, serta evaluasi untuk memperoleh model dengan performa terbaik.

7. Integrasi Edge AI

Model terbaik hasil pelatihan dikonversi ke format ONNX dan diintegrasikan ke dalam aplikasi Android menggunakan ONNX Runtime sehingga proses inferensi dapat dijalankan secara lokal (*on-device inference*) pada perangkat.

8. Pengujian Sistem di Lapangan

Tahap akhir dilakukan untuk menguji aplikasi yang telah terintegrasi dengan model *Deep Learning* pada kondisi penggunaan sebenarnya. Pengujian bertujuan mengevaluasi kemampuan sistem dalam melakukan deteksi anomali

permukaan jalan secara *real-time* serta memastikan implementasi *Edge AI* dapat berjalan sesuai dengan kebutuhan penelitian.

3.2 Desain Penelitian

3.2.1. Peralatan Penelitian

Penelitian ini memanfaatkan perangkat keras dan perangkat lunak untuk mendukung proses akuisisi data, pelatihan model, implementasi sistem, dan pengujian aplikasi. Rincian peralatan yang digunakan dalam penelitian ini disajikan pada Tabel 3.1.

Tabel 3.III.1 Peralatan Penelitian

Kategori	Komponen	Spesifikasi / Keterangan
Perangkat Keras	Komputer Pelatihan Model	Prosesor Intel Core i3 1115G4, RAM 16 GB, GPU Intel(R) UHD Graphic.
	Perangkat Seluler	Smartphone Android yang dilengkapi sensor akselerometer, giroskop, dan GPS.
	Kendaraan Uji	Sepeda motor tipe skuter matik 125cc.
Perangkat Lunak	Lingkungan Pelatihan	Windows 11, Python 3.10+, PyTorch, Scikit-Learn, Pandas, dan NumPy.
	Lingkungan Pengembangan	Android Studio, Kotlin, Room Database (SQLite), ONNX Runtime Android, dan Google Maps SDK.

3.2.2. Kebutuhan Data

Penelitian ini menggunakan data primer yang diperoleh melalui proses akuisisi menggunakan sensor bawaan pada smartphone Android. Data yang dikumpulkan berupa data *multivariate time-series* yang terdiri atas sinyal akselerometer triaksial, giroskop triaksial, serta informasi lokasi dan kecepatan kendaraan yang diperoleh dari sensor GPS. Proses pengambilan data dilakukan

pada berbagai kondisi permukaan jalan beraspal di wilayah Kabupaten Pematang.

Dataset yang dihasilkan digunakan untuk membangun model klasifikasi dengan tiga kelas, yaitu *Pothole* (lubang jalan), *Speed Bump* (polisi tidur), dan *Non-Event* (kondisi jalan normal). Data tersebut selanjutnya diproses melalui tahapan pra-pemrosesan, pelatihan model, evaluasi, hingga implementasi pada sistem deteksi anomali permukaan jalan berbasis Edge AI.

3.3 Akuisisi dan Pra-Pemrosesan Data

Tahap ini bertujuan menghasilkan dataset yang siap digunakan pada proses pelatihan model 1D-CNN. Proses dimulai dari pengembangan aplikasi akuisisi data, pengambilan data sensor di lapangan, pelabelan data berdasarkan kejadian anomali, hingga tahapan pra-pemrosesan untuk membentuk tensor masukan model.

3.3.1. Pengembangan Aplikasi Data Logger

Tahap awal penelitian dilakukan dengan mengembangkan aplikasi Android yang berfungsi sebagai *data logger* untuk merekam data sensor selama proses akuisisi di lapangan. Aplikasi dikembangkan menggunakan bahasa pemrograman Kotlin dan dirancang untuk memperoleh data sensor akselerometer, giroskop, dan GPS secara simultan. Data yang direkam meliputi *timestamp*, akselerometer tiga sumbu (A_x , A_y , A_z), giroskop tiga sumbu (G_x , G_y , G_z), koordinat geografis (*latitude* dan *longitude*), serta kecepatan kendaraan (*speed*).

Pada penelitian ini, koordinat GPS tidak digunakan sebagai masukan model klasifikasi, melainkan dimanfaatkan sebagai referensi spasial untuk proses pelabelan data dan visualisasi hasil deteksi. Sementara itu, informasi kecepatan kendaraan digunakan sebagai salah satu kanal masukan model bersama data akselerometer dan giroskop.

3.3.2. Akuisisi Data Lapangan

Proses akuisisi data dilakukan menggunakan smartphone Android yang dipasang pada *holder* stang kendaraan dengan orientasi tegak (*portrait*)

menghadap ke pengemudi. Pengambilan data dilakukan menggunakan kendaraan roda dua tipe skuter matik dengan sistem suspensi standar bawaan pabrik sesuai dengan batasan penelitian. Posisi pemasangan smartphone selama proses pengambilan data ditunjukkan pada Gambar 3.2.



Gambar 3.III.2 Posisi Smartphone

Perekaman dilakukan pada berbagai ruas jalan beraspal di wilayah Kabupaten Pematang Jaya yang merepresentasikan tiga kondisi permukaan jalan, yaitu Pothole, Speed Bump, dan Non-Event. Selama proses perekaman, sensor akselerometer dan giroskop digunakan untuk merekam respons getaran kendaraan terhadap kondisi permukaan jalan, sedangkan sensor GPS digunakan untuk merekam informasi koordinat geografis dan kecepatan kendaraan secara simultan.

3.3.3. Pra-Pemrosesan Sinyal

Sebelum diekstraksi menjadi potongan-potongan sampel untuk pembelajaran mesin, data kontinu pada masing-masing perjalanan harus dibersihkan terlebih dahulu melalui operasi pemrosesan sinyal digital. Tahapan ini mencakup tiga operasi utama yang dilakukan secara berurutan.

Operasi pertama adalah penyampelan ulang (*resampling*) yang bertujuan untuk menyeragamkan frekuensi pencuplikan data menjadi konstan pada 100

Hz, sehingga setiap baris observasi memiliki interval waktu mutlak sebesar 10 ms. Proses ini dieksekusi menggunakan metode interpolasi linier. Guna menjaga keaslian karakteristik sinyal dan mencegah pembentukan data sintetis yang berlebihan, rentang interpolasi dibatasi maksimal pada lima sampel berturut-turut (*limit* = 5). Pembatasan ini memastikan bahwa jika terjadi kehilangan data yang melebihi durasi 50 ms, sistem tidak akan memaksa untuk merekonstruksi sinyal yang hilang secara semu.

Operasi kedua adalah penanganan kekosongan sinyal (*gap handling*). Berkaitan dengan batasan interpolasi sebelumnya, algoritma mendeteksi anomali transmisi perangkat keras sensor yang menyebabkan terjadinya waktu kosong (*dropped frames*) berskala besar. Segmen sinyal yang berada di sekitar area kekosongan transmisi tersebut diisolasi dari proses ekstraksi dataset. Tindakan karantina ini krusial untuk menjamin bahwa urutan temporal sinyal (*temporal contiguity*) yang diumpukan ke model selalu valid secara fisik.

Operasi ketiga dan terakhir adalah fusi sensor (*sensor fusion*) dan pemfilteran sinyal. Proses ini memanfaatkan luaran fusi perangkat keras bawaan (*hardware sensor fusion*) perangkat, yaitu percepatan linear (*lin_ax*, *lin_ay*, *lin_az*) dan vektor gravitasi. Sinyal percepatan linear diproyeksikan terhadap vektor gravitasi untuk memisahkan komponen percepatan vertikal (*a_vertical*) dan horizontal (*a_horizontal*). Sinyal vertikal kemudian dilewatkan pada penyaring pita-lewat (*Butterworth Bandpass Filter* 1–20 Hz) untuk mereduksi derau getaran mesin dan pergeseran dasar (*DC drift*). Sementara itu, sinyal horizontal dilewatkan pada penyaring lolos-bawah (*Butterworth Low-Pass Filter* 20 Hz) guna meredam derau tanpa merusak nilai dasar absolutnya. Kombinasi dari kedua komponen yang telah difilter tersebut dihitung magnitudo vektornya untuk menghasilkan fitur akhir *a_linear_mag*.

3.3.4. Pelabelan Data

Pelabelan data dilakukan menggunakan data hasil proses pra-pemrosesan sinyal. Proses pelabelan data menggunakan pendekatan kombinasi deteksi kandidat kejadian secara otomatis dengan proses verifikasi manual oleh peneliti

untuk membentuk *ground truth*. Pada tahap ini, rekaman suara tidak diproses secara otomatis oleh sistem, melainkan hanya digunakan sebagai referensi selama proses pelabelan.

Tahap pertama dilakukan dengan menerapkan algoritma *peak detection* pada sinyal akselerasi yang telah melalui tahap pra-pemrosesan sinyal. Tahapan ini menghasilkan sinyal yang digunakan sebagai dasar untuk mendeteksi kandidat kejadian (*event candidate*) berdasarkan perubahan karakteristik getaran kendaraan. Setiap kandidat kejadian yang memenuhi kriteria deteksi ditampilkan pada antarmuka pelabelan berbasis Folium dalam bentuk *marker* yang dilengkapi informasi waktu kejadian dan lokasi geografis

Selanjutnya, peneliti mendengarkan kembali rekaman suara untuk mengetahui waktu ketika kejadian anomali terjadi dan diucapkan, kemudian mencocokkannya dengan informasi waktu yang ditampilkan pada antarmuka pelabelan. Berdasarkan kecocokan tersebut, setiap kandidat kejadian diverifikasi secara manual dan diberi salah satu dari tiga label, yaitu *Pothole*, *Speed Bump*, atau *Non-Event*. Hasil akhir proses ini berupa kumpulan *timestamp* kejadian yang telah tervalidasi beserta label kelasnya, yang selanjutnya digunakan pada proses pembentukan dataset pelatihan.

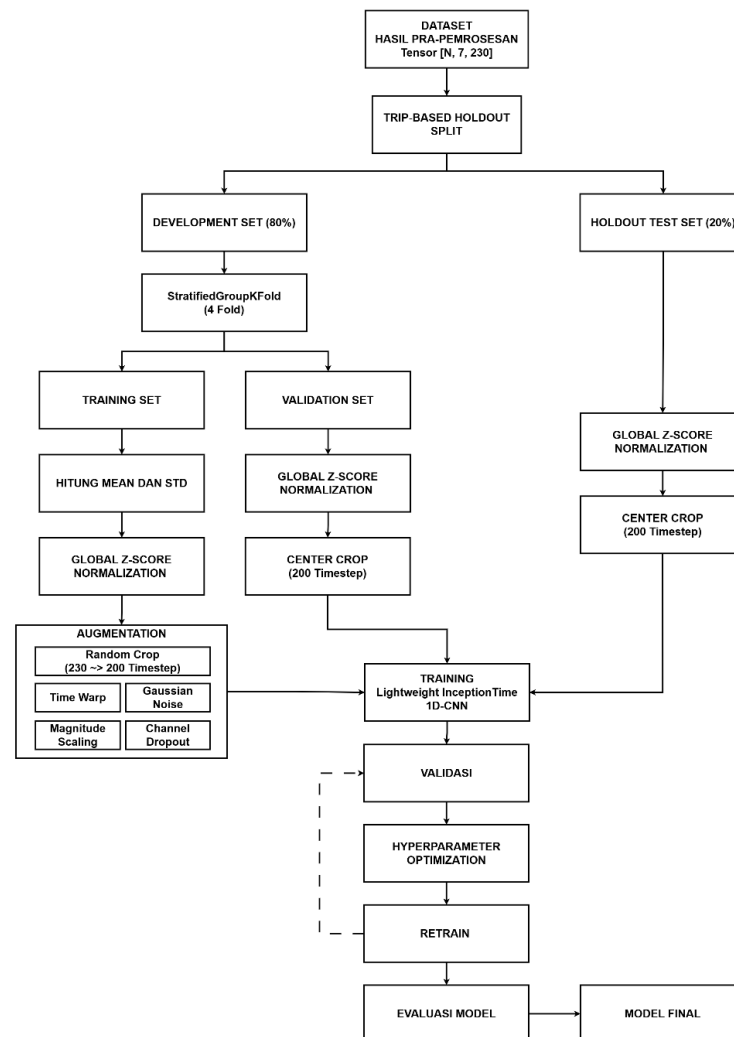
3.3.5. Pembentukan Dataset

Pada tahapan ini, data kontinu yang telah dikurasi dan dilabeli ditransformasikan secara struktural ke dalam bentuk tensor multidimensi (berdimensi $[N, 7, 230]$) yang kompatibel untuk dikonsumsi oleh arsitektur *Deep Learning*. Tahapan ini difokuskan secara eksklusif pada Ekstraksi Urutan (*Sequence Extraction*): Setiap titik pusat kejadian (*event*) yang berlabel diperluas menjadi sebuah jendela sinyal (*signal window*) berdurasi 2,3 detik, yang secara matematis setara dengan 230 *timestep* pada frekuensi 100 Hz. Matriks-matriks potongan sinyal berdimensi spasial dan temporal dari seluruh entitas perjalanan (*trip*) ini kemudian ditumpuk (*stacked*) dan ditransposisi menjadi satu himpunan dataset statis utuh. Tahapan

operasional ini merupakan titik terminasi (akhir) dari alur persiapan bentuk data.

3.4 Metode Yang Diusulkan

Tahap pemodelan dilakukan untuk membangun model klasifikasi yang digunakan dalam mendeteksi anomali permukaan jalan berdasarkan data sensor smartphone. Proses ini meliputi pembagian dataset, perancangan arsitektur model, strategi pelatihan, serta penyetelan hyperparameter agar diperoleh konfigurasi model yang paling sesuai dengan karakteristik data. Alur pemodelan pada penelitian ini ditunjukkan pada Gambar 3.5.



Gambar 3.3 Diagram Alur Pemodelan

3.4.1. Pembagian Dataset

Untuk menjamin pengujian model yang representatif terhadap kondisi operasional di lapangan, kumpulan data yang telah terbentuk dipartisi menjadi dua kelompok utama, yaitu himpunan latih (*Development Set*) sebesar 80% dan himpunan uji mutlak (*Holdout Test Set*) sebesar 20%. Pemisahan data ini dieksekusi secara ketat menggunakan pendekatan *Trip-Based Holdout Split*. Pemisahan berbasis *trip* memastikan bahwa sampel guncangan yang berasal dari lintasan perjalanan yang sama tidak akan terdistribusi melintasi batas partisi. Hal ini krusial untuk mencegah terjadinya kebocoran informasi temporal (*data leakage*) yang berpotensi menghasilkan estimasi performa yang terlalu optimistis.

Khusus pada *Development Set*, metode Validasi Silang Grup Berstrata (*Stratified Group K-Fold Cross-Validation*, $k=4$) diaplikasikan guna mengevaluasi kinerja arsitektur secara lebih stabil dan objektif. Metode validasi ini mendistribusikan *trip* ke dalam empat lipatan sedemikian rupa sehingga proporsi kelas tetap terjaga keseimbangannya pada setiap iterasi pengujian tanpa memecah satu perjalanan ke lebih dari satu lipatan.

3.4.2. Normalisasi

Fitur sensor yang digunakan sebagai masukan model memiliki rentang skala amplitudo yang sangat bervariasi. Untuk mereduksi variabilitas data dan mempercepat proses konvergensi, teknik Global Z-Score Normalization diimplementasikan guna menstandarisasi distribusi data agar memiliki nilai rata-rata nol dan simpangan baku satu.

Guna menjaga integritas metodologi eksperimen, perhitungan parameter normalisasi dikerjakan secara komputasional dinamis. Nilai rata-rata dan simpangan baku dihitung secara eksklusif dari himpunan latih pada setiap iterasi lipatan (*train fold*). Parameter tersebut kemudian dikunci dan digunakan untuk menstandarisasi partisi validasinya (*val fold*). Perhitungan dinamis ini didesain untuk memastikan tidak ada informasi distribusi statistik dari data pengujian yang bocor kembali ke dalam fase pelatihan. Setelah proses pelatihan

silang selesai, parameter normalisasi dihitung kembali menggunakan seluruh populasi *Development Set* sebagai basis untuk pelatihan model final dan inferensi *on-device*.

3.4.3. Augmentasi Data

Data kondisi permukaan jalan di dunia nyata sarat akan variasi yang sulit diprediksi, mulai dari perbedaan kecepatan pengemudi hingga latensi perangkat keras *smartphone*. Untuk meredam sensitivitas model terhadap variasi tersebut dan mencegah penghafalan data latih (*overfitting*), mekanisme augmentasi data yang dikalkulasi secara dinamis (*Dynamic Jitter*) diaplikasikan secara *on-the-fly* pada fase pelatihan. Rincian lima transformasi augmentasi yang diterapkan disajikan pada Tabel 3.2.

Tabel III.2 Parameter Augmentasi Data (Dynamic Jitter)

Transformasi	Mekanisme Komputasi	Tujuan Simulasi Klinis
<i>Random Cropping</i> ($230 \rightarrow 200$ timestep)	Pemotongan window berukuran 230 timestep menjadi 200 timestep pada titik acak.	Mensimulasikan jeda latensi perangkat (time jittering).
<i>Time Warping</i> ($time_warp_mag = 0.1$)	Deformasi kurva waktu non-linear melalui titik kontrol acak.	Mereplikasikan variasi kecepatan laju kendaraan.
<i>Gaussian Noise</i> ($noise_std = 0.02$)	Injeksi fluktuasi derau berdistribusi normal pada seluruh kanal.	Mensimulasikan gangguan mekanik sekunder (vibrasi mesin).
<i>Magnitude Scaling</i> ($scale_range = (0.85, 1.15)$)	Pengalian amplitudo sinyal dengan faktor skala independen acak.	Mensimulasikan perbedaan sensitivitas kalibrasi antar smartphone.

Transformasi	Mekanisme Komputasi	Tujuan Simulasi Klinis
<i>Channel Dropout</i> (<i>channel_drop_prob</i> = 0.1)	Penonaktifan nilai pada satu sumbu sensor menjadi nol secara acak.	Mensimulasikan kegagalan bacaan sensor sesaat.

Seluruh transformasi augmentasi pada Tabel 3.1 hanya diaktifkan pada himpunan pelatihan. Pada data validasi dan pengujian, augmentasi dinonaktifkan, dan *window* dipotong secara presisi di tengah (*center crop*) untuk menjaga konsistensi representasi sinyal evaluasi.

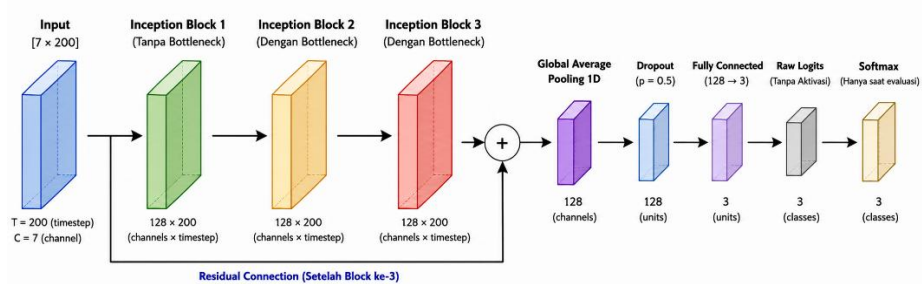
3.4.4. Penanganan Ketidakseimbangan Kelas

Distribusi data kejadian permukaan jalan secara inheren didominasi oleh kondisi kelas *Non-Event*, yang memicu terjadinya ketidakseimbangan kelas (*class imbalance*) ekstrem terhadap anomali jalan. Jika menggunakan fungsi evaluasi standar, model akan cenderung menebak kelas mayoritas secara aman demi mencapai metrik akurasi yang ilusif.

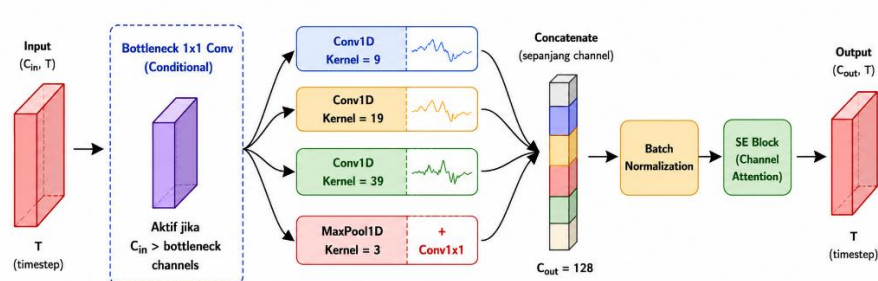
Untuk memitigasi kendala struktural ini, fungsi pengukuran kerugian entropi silang konvensional direkonstruksi menjadi *Multi-Class Focal Loss*. Fungsi objektif ini didesain secara matematis dengan menyuntikkan faktor pemfokusan (*focusing parameter*). Melalui formulasi ini, skala bobot gradien diturunkan secara drastis pada prediksi kelas mayoritas yang amat mudah ditebak, sehingga algoritma optimisasi dipaksa untuk memusatkan perhatian pada sampel kelas minoritas yang sulit diklasifikasikan (*hard examples*). Implementasi arsitektur jaringan pada penelitian ini tidak menggunakan penambahan bobot kelas (*class weight*) secara eksplisit, melainkan mengandalkan fungsionalitas murni dari mekanisme *focusing* bawaan *Focal Loss*.

3.4.5. Arsitektur 1D-CNN

Model yang digunakan pada penelitian ini merupakan arsitektur *Lightweight InceptionTime 1D-CNN* yang dimodifikasi dengan penambahan *Squeeze-and-Excitation (SE) Block* pada setiap blok *Inception*. Model dirancang untuk memproses data runtun waktu multikanal berukuran $[7 \times 200]$, yang terdiri atas tiga kanal akselerometer, tiga kanal giroskop, dan satu kanal kecepatan kendaraan. Arsitektur model secara keseluruhan ditunjukkan pada Gambar 3.6.



Gambar III.4 Arsitektur Lightweight InceptionTime 1D-CNN



Gambar III.5 Detail Struktur Inception Block

Setiap *InceptionBlock1D* dilengkapi mekanisme bottleneck berupa konvolusi 1×1 yang bekerja secara kondisional. Mekanisme bottleneck hanya diterapkan ketika jumlah kanal masukan blok tersebut lebih besar dari jumlah kanal target bottleneck. Pada blok pertama, kanal masukan berjumlah 7 sesuai jumlah kanal sensor, sehingga blok ini memproses seluruh kanal secara langsung tanpa reduksi terlebih dahulu. Mekanisme bottleneck kemudian aktif pada blok berikutnya ketika jumlah kanal masukan telah meningkat menjadi

128, sehingga kanal fitur direduksi terlebih dahulu sebelum masuk ke cabang konvolusi paralel. Pendekatan ini menjaga efisiensi komputasi tanpa menghilangkan informasi penting pada tahap awal ekstraksi fitur dari sinyal mentah.

Setiap InceptionBlock1D terdiri atas beberapa cabang paralel, yaitu tiga cabang *Conv1D* dengan ukuran kernel berbeda, yaitu 9, 19, dan 39, serta satu cabang *MaxPooling*. Kombinasi cabang multi-skala ini memungkinkan model menangkap pola sinyal pada rentang waktu yang berbeda, mulai dari impuls singkat hingga getaran yang lebih panjang. Keluaran dari seluruh cabang kemudian digabungkan menggunakan operasi *concatenation* dan diproses menggunakan *Batch Normalization* untuk menstabilkan distribusi aktivasi selama pelatihan. Selanjutnya, setiap blok Inception dilengkapi dengan *SE Block* sebagai mekanisme *channel attention* yang berfungsi memberikan bobot secara adaptif pada kanal fitur yang paling relevan.

Model tersusun atas tiga blok Inception, dengan hasil keluaran dihubungkan menggunakan *residual connection* setelah blok ketiga untuk menjaga aliran gradien selama proses pelatihan. Pada bagian akhir jaringan, fitur yang telah diekstraksi diringkas menggunakan *Global Average Pooling* sebagai satu-satunya tahap reduksi dimensi temporal pada keseluruhan arsitektur. Setelah itu, data diteruskan ke *Fully Connected Layer* untuk menghasilkan tiga nilai keluaran berupa *raw logits* yang merepresentasikan kelas *Pothole*, *Speed Bump*, dan *Non-Event*. Model tidak menggunakan fungsi aktivasi pada lapisan keluaran. Selama proses pelatihan, *raw logits* diproses menggunakan *Multi-Class Focal Loss*, sedangkan pada tahap evaluasi fungsi *Softmax* digunakan untuk mengubah keluaran model menjadi distribusi probabilitas antar kelas.

3.4.6. Konfigurasi Pelatihan Baseline

Tahapan eksperimentasi dimulai dengan pelatihan menggunakan parameter operasional *baseline*. Proses ini krusial untuk membuktikan sejauh mana kapabilitas murni arsitektur *InceptionTime* dalam memisahkan kelas

anomali jalan sebelum dilakukannya intervensi algoritma pencarian lanjutan. Rincian konfigurasi pelatihan yang ditetapkan pada arsitektur dasar dijabarkan pada Tabel 3.3.

Tabel III.3 Konfigurasi Baseline Arsitektur InceptionTime 1D-CNN

Komponen Pelatihan	Konfigurasi Operasional (Nilai <i>Default</i>)
<i>Base Channels</i>	128 Kanal
<i>Dropout Rate</i>	0,5
<i>Batch Size</i>	32
<i>Gamma (Focal Loss)</i>	2,0
<i>Initial Learning Rate</i>	0,001
<i>Weight Decay</i>	0,0001
<i>Optimizer</i>	Adam
<i>Learning Rate Scheduler</i>	<i>CosineAnnealingLR</i> (hingga nilai minimum 1×10^{-6})
<i>Gradient Clipping</i>	<i>Max Gradient Norm</i> = 1.0
<i>Checkpointing</i>	<i>Best Model Checkpointing</i> berdasarkan <i>PR-AUC</i>

Penggunaan mekanisme penjadwalan *CosineAnnealingLR* diformulasikan untuk melakukan peluruhan laju pembelajaran secara mulus menuju akhir proses pelatihan, sementara *Gradient Clipping* diimplementasikan untuk mencegah ledakan gradien. Pada skema baseline ini, seluruh parameter ditetapkan secara empiris heuristik sebelum adanya intervensi algoritma tuning. Mekanisme *Best Model Checkpointing* secara otomatis menyimpan bobot model terbaik dari setiap lipatan validasi silang berdasarkan metrik PR-AUC.

3.4.7. Hyperparameter Tuning

Tahapan penyetelan *hyperparameter* diimplementasikan untuk memperoleh konfigurasi arsitektur yang menghasilkan performa mutlak pada data pengembangan. Proses ini dilaksanakan secara otonom menggunakan framework Optuna. Untuk menjaga efisiensi waktu komputasi, algoritma

sampler berbasis *Tree-structured Parzen Estimator* (TPE) dipadukan dengan metode *MedianPruner* (5 iterasi awal pemanasan) guna menghentikan komputasi eksperimen yang terindikasi tidak menjanjikan secara dini. Rincian konfigurasi ruang pencarian disajikan pada Tabel 3.3.

Tabel 3.4 Ruang Pencarian Hyperparameter (Optuna)

Parameter Optimisasi	Rentang Pencarian / Kandidat	Skala / Tipe Distribusi
<i>Learning Rate</i>	1×10^{-5} hingga 1×10^{-1}	Skala Logaritmik
<i>Weight Decay</i>	1×10^{-5} hingga 1×10^{-2}	Skala Logaritmik
<i>Dropout Rate</i>	0,1 hingga 0,7	Skala Linear
<i>Gamma (Focal Loss)</i>	1,0 hingga 3,0	Skala Linear
<i>Batch Size</i>	{16, 32, 64, 128}	Kandidat Kategorikal
<i>Base Channels</i>	{16, 32, 64}	Kandidat Kategorikal

Optimisasi dijalankan sebanyak 50 percobaan (trial). Pada tiap epoch, sistem mengekstraksi nilai rata-rata Macro F1-Score dari seluruh lipatan validasi sebagai landasan evaluasi. Agar lintasan perburuan algoritma tidak terjebak pada metrik akurasi semu, fungsi objektif akhir yang dimaksimalkan adalah tingkat ketahanan model (robust score). Metrik ini dihitung secara eksak melalui selisih antara nilai rata-rata Macro F1-Score dan simpangan bakunya (*mean - standard deviation*) di antara seluruh lipatan uji, memastikan konfigurasi yang dipilih stabil di segala kondisi.

3.5 Evaluasi Model

Evaluasi model dilakukan untuk mengukur performa arsitektur *ID-CNN* dalam memprediksi anomali permukaan jalan. Evaluasi dilaksanakan menggunakan beberapa metrik evaluasi klasifikasi, seperti *Confusion*

Matrix, Precision, Recall, Macro F1-Score, dan PR-AUC (Precision-Recall Area Under Curve).

Penggunaan berbagai metrik evaluasi ini bertujuan untuk memberikan gambaran performa model secara menyeluruh dan objektif. Dalam konteks klasifikasi dengan distribusi kelas yang sangat tidak seimbang (*imbalanced data*), ketergantungan pada metrik akurasi konvensional dapat menghasilkan estimasi performa ilusif yang didominasi oleh tebakan kelas mayoritas (*Non-Event*). Oleh karena itu, *Macro F1-Score* dan *PR-AUC* menjadi metrik yang sangat krusial untuk mengevaluasi ketahanan model pada kelas minoritas (*Pothole* dan *Speed Bump*). Hasil evaluasi dari masing-masing skenario pengujian selanjutnya dianalisis untuk menentukan konfigurasi metode yang memiliki performa klasifikasi terbaik.

3.6 Integrasi Edge AI dan Pengujian Akhir

Tahap ini dilakukan untuk mengintegrasikan model yang telah dilatih ke dalam perangkat Android sehingga proses inferensi dapat dijalankan secara lokal (*on-device inference*). Setelah model terbaik diperoleh dari proses pelatihan, model tersebut diekspor ke format ONNX (*Open Neural Network Exchange*) melalui skrip terpisah agar dapat digunakan pada lingkungan *mobile*.

3.6.1. Konversi Model ONNX

Model terbaik yang diperoleh dari proses pelatihan disimpan dalam format state dictionary PyTorch. Pada skrip pelatihan, ekspor ONNX tidak dilakukan secara langsung, melainkan dipindahkan ke skrip terpisah agar proses ekspor dapat menerapkan pembungkus inferensi yang sesuai untuk perangkat bergerak. Model dasar terlebih dahulu direkonstruksi menggunakan bobot hasil pelatihan model final. Selanjutnya dibangun sebuah modul *MobileInferenceWrapper* yang menggabungkan tiga komponen utama, yaitu parameter normalisasi *Global Z-Score*, model klasifikasi *InceptionTime1D*, dan fungsi Softmax sebagai keluaran probabilitas.

Dengan pendekatan ini, seluruh proses prapemrosesan yang diperlukan pada saat inferensi telah dibekukan (*embedded*) ke dalam grafik komputasi

ONNX. Akibatnya, aplikasi Android tidak lagi perlu menghitung normalisasi maupun transformasi *Softmax* secara terpisah, tetapi cukup memberikan masukan berupa tensor murni hasil pembentukan *window*.

3.6.2. Perancangan Aplikasi Android

Aplikasi final dikembangkan untuk menjalankan proses inferensi secara lokal pada perangkat Android. Aplikasi menerima data sensor yang telah diproses sebelumnya, menjalankan model ONNX melalui *runtime* yang sesuai, lalu menampilkan hasil prediksi kondisi jalan secara langsung pada layar.

Sistem aplikasi ini mengadopsi pola arsitektur perangkat lunak Model-View-ViewModel (MVVM) untuk memisahkan antarmuka pengguna dari logika pemrosesan data, serta menggunakan *Room Database* sebagai basis data lokal. Komponen fungsional utama dari aplikasi dijabarkan secara rinci pada Tabel 3.5.

Tabel III.4 Komponen Fungsional Android

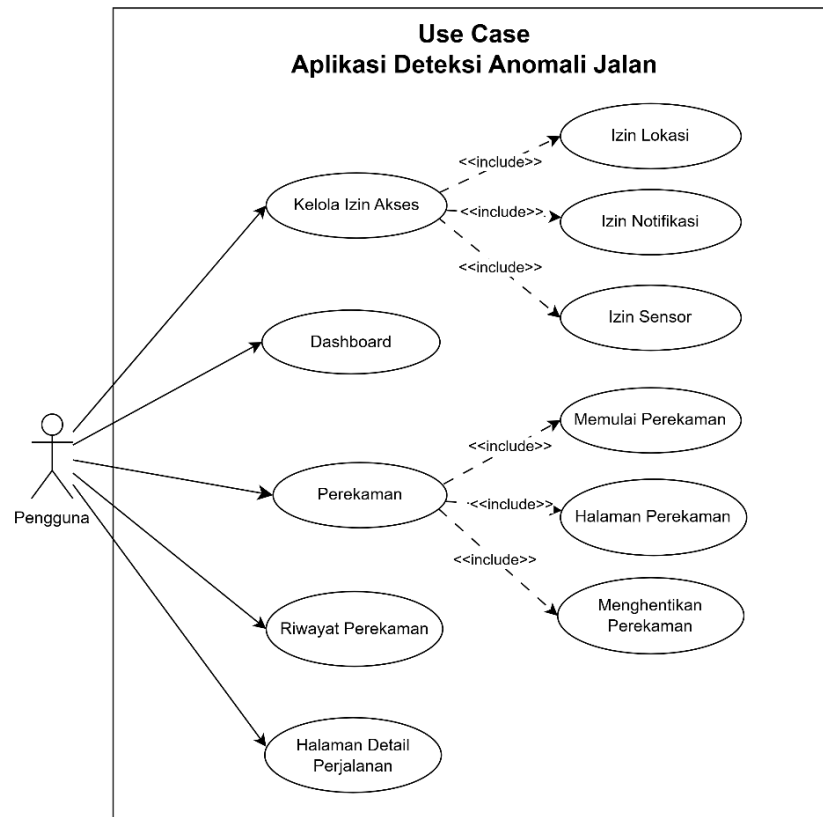
Komponen Fungsional	Deskripsi Mekanisme Teknis
<i>Sensor Acquisition</i>	Beroperasi di dalam <i>Foreground Service</i> untuk merekam aliran data akselerometer dan giroskop. Proses akuisisi dilakukan secara asinkron dengan <i>oversampling</i> pada frekuensi 200 Hz.
<i>Sensor Fusion Processor</i>	Memotong aliran data menjadi <i>sliding window</i> berukuran 200 <i>timestep</i> (pasca- <i>resampling</i> 100 Hz). Inferensi akan dilewati sepenuhnya secara otomatis apabila laju kendaraan berada di bawah 1,0 m/s demi meminimalkan <i>false positive</i> .
<i>ONNX Runtime Module</i>	Memuat artefak model klasifikasi yang telah diekspor dan mengeksekusi inferensi secara lokal (<i>on-device</i>). Modul ini menerima masukan tensor spasio-temporal berdimensi $[1 \times 7 \times 200]$ dan memproyeksikan probabilitas kejadian.

Komponen Fungsional	Deskripsi Mekanisme Teknis
<i>Local Storage (Room DB)</i>	Basis data SQLite yang secara asinkron menyimpan riwayat perjalanan spasial serta lokasi persis kejadian anomali jalan (<i>Pothole</i> dan <i>Speed Bump</i>).
<i>Map Visualization</i>	Mengintegrasikan <i>Google Maps API</i> untuk merender titik-titik anomali sebagai <i>marker</i> pada antarmuka peta digital.

Untuk mengorkestrasikan seluruh komponen fungsional tersebut menjadi sebuah aplikasi *mobile* yang terpadu, diperlukan rancangan pemodelan sistem yang komprehensif. Pemodelan arsitektur perangkat lunak ini dirancang untuk memvisualisasikan struktur interaksi pengguna, alur logika inferensi data sensor, serta skema basis data lokal secara terstruktur. Tahapan perancangan perangkat lunak ini secara sekuensial diawali dengan mendefinisikan ruang lingkup fungsionalitas dan interaksi aktor sistem melalui penjabaran *Use Case Diagram*.

a. Diagram Use Case

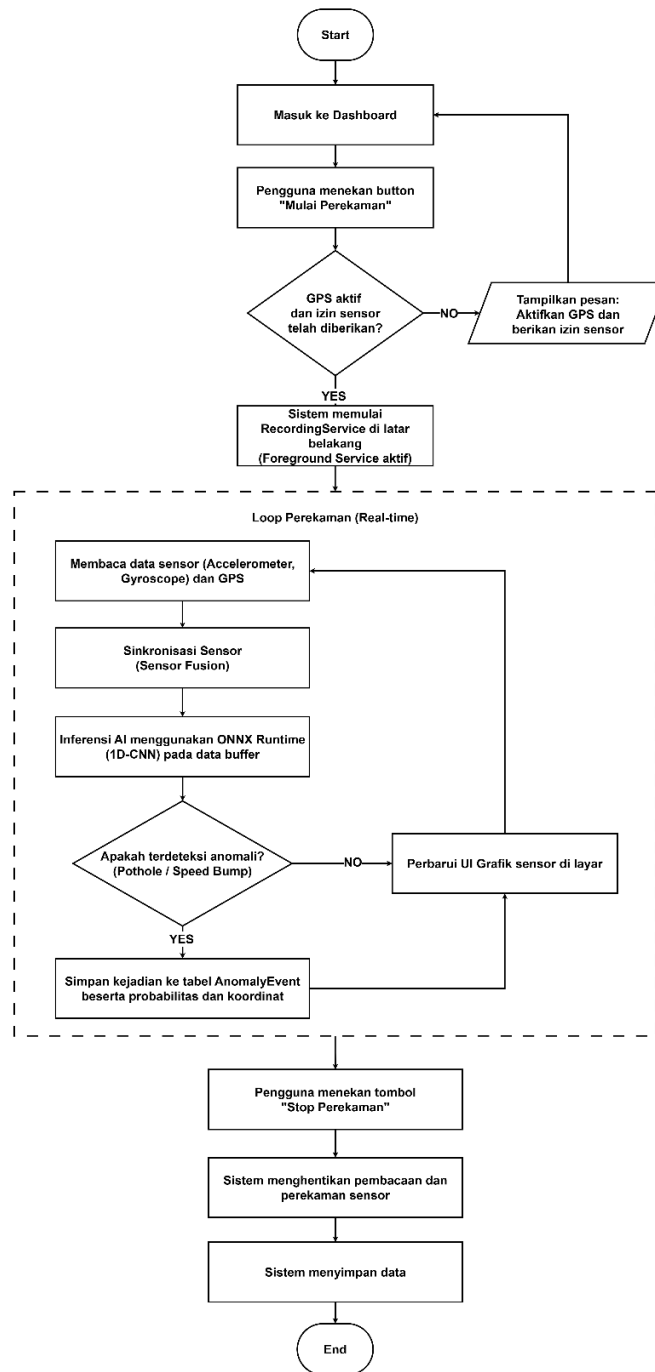
Diagram *Use Case* menggambarkan interaksi antara aktor utama, yaitu pengemudi, dengan sistem aplikasi. Diagram ini menunjukkan fungsi-fungsi utama yang tersedia, mulai dari memulai perjalanan, menghentikan perekaman, melakukan inferensi, melihat hasil deteksi pada peta, hingga menyimpan riwayat perjalanan.



Gambar III.6 Diagram Use Case Aplikasi Deteksi Anomali Jalan

b. Flowchart Alur Akuisisi & Inferensi Edge AI

Flowchart ini menggambarkan alur pemrosesan data sensor yang berjalan secara berurutan di dalam Foreground Service. Proses dimulai dari akuisisi data sensor, pembentukan *sliding window*, inferensi menggunakan model ONNX, hingga penyimpanan hasil deteksi dan visualisasi pada peta.

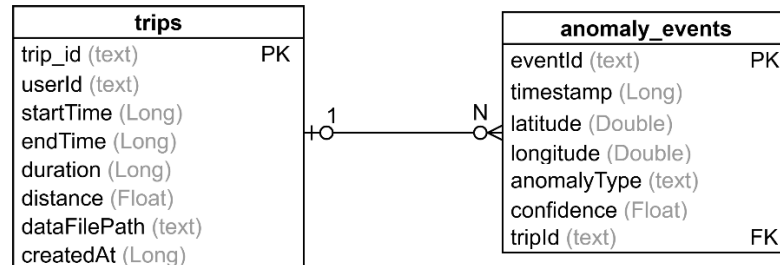


Gambar III.7 Diagram Flowchart Alur Akuisisi dan Inferensi

c. Entity Relationship Diagram (ERD) Room Database

Entity Relationship Diagram (ERD) digunakan untuk menggambarkan struktur penyimpanan data lokal menggunakan Room Database. Basis data ini menyimpan informasi perjalanan (*trip*) beserta hasil

deteksi anomali yang diperoleh selama proses inferensi sehingga aplikasi tetap dapat beroperasi tanpa koneksi internet.



Gambar III.8 Diagram ERD Aplikasi

3.6.3. Skenario Pengujian Lapangan

Pengujian lapangan dilakukan untuk memastikan bahwa sistem yang telah diintegrasikan pada perangkat Android dapat beroperasi pada kondisi nyata. Tahap ini berfokus pada kemampuan sistem dalam melakukan inferensi secara lokal, menghasilkan prediksi yang konsisten, serta tetap berjalan stabil selama pengujian di lapangan. Hasil pengujian kemudian digunakan untuk menilai keberhasilan integrasi model ke dalam aplikasi serta kesesuaian sistem dengan kebutuhan pemantauan kondisi jalan secara real-time.

Pada skenario ini, pengujian tidak hanya menilai keluaran klasifikasi, tetapi juga memverifikasi bahwa proses inferensi dapat dilakukan tanpa bergantung pada koneksi internet. Dengan demikian, pengujian lapangan menjadi tahap akhir untuk membuktikan bahwa model yang telah dibangun benar-benar dapat digunakan sebagai komponen Edge AI pada perangkat bergerak.

BAB IV

HASIL PENELITIAN DAN PEMBAHASAN

4.1 Implementasi Aplikasi Akuisisi Data

Sebagai tahap awal penelitian, berhasil dikembangkan sebuah aplikasi Android yang berfungsi sebagai Data Logger untuk mengakuisisi data sensor selama proses pembentukan dataset. Aplikasi ini digunakan sebagai media perekaman data akselerometer, giroskop, GPS, dan kecepatan kendaraan secara kontinu selama kendaraan bergerak. Berbeda dengan aplikasi akhir yang digunakan pada tahap deployment, aplikasi pada tahap ini belum mengintegrasikan model kecerdasan buatan dan hanya difokuskan pada proses akuisisi data sensor.

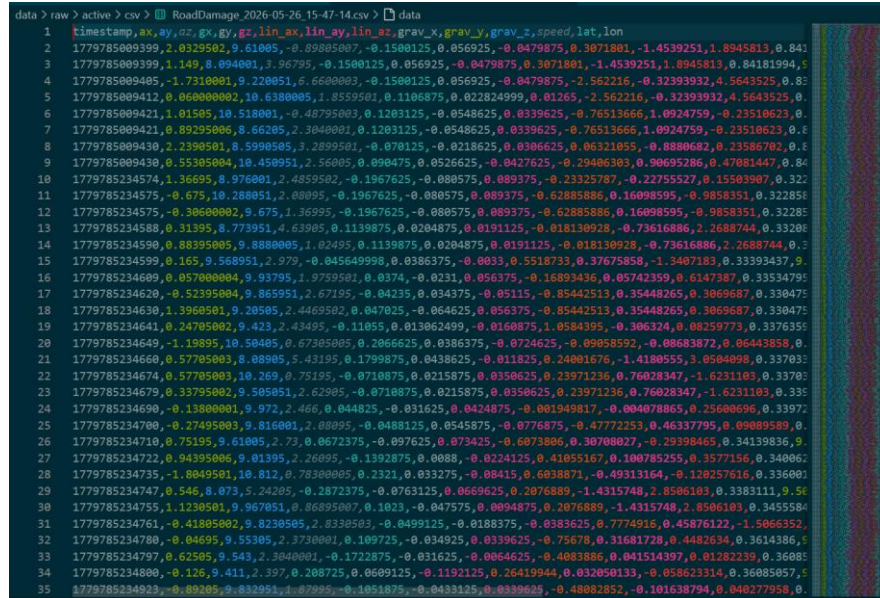
Hasil implementasi aplikasi ditunjukkan pada Gambar 4.1. Antarmuka aplikasi menyediakan fungsi utama untuk memulai dan menghentikan proses perekaman (*Start Recording* dan *Stop Recording*), menampilkan status sensor, serta memberikan informasi mengenai proses akuisisi yang sedang berlangsung. Seluruh data sensor yang diperoleh selama perjalanan disimpan dalam berkas berformat *.csv* sehingga dapat digunakan pada tahapan pembentukan dataset.

Gambar 4.1. Antarmuka aplikasi *Data Logger*

Implementasi aplikasi memanfaatkan mekanisme *Foreground Service* sehingga proses pembacaan sensor tetap berlangsung meskipun aplikasi berada pada latar belakang. Selama pengujian lapangan, mekanisme ini mampu mempertahankan proses akuisisi data secara kontinu tanpa terganggu oleh manajemen proses sistem operasi Android. Dengan demikian, data sensor yang diperoleh memiliki kontinuitas yang baik dan dapat digunakan sebagai dasar pembentukan dataset penelitian.

Hasil perekaman ditampilkan dalam bentuk berkas data mentah yang memuat informasi *timestamp*, akselerometer tiga sumbu (A_x , A_y , A_z), giroskop tiga sumbu (G_x , G_y , G_z), koordinat GPS (*latitude* dan *longitude*), serta kecepatan kendaraan

(*speed*). Penyimpanan seluruh informasi tersebut dalam satu berkas memungkinkan sinkronisasi temporal antar sensor tetap terjaga selama proses pra-pemrosesan data.



```
data > raw > active > csv > RoadDamage_2026-05-26_15-47-14.csv > data
1 timestamp,ax,ay,az,gx,gy,gz,lin_ax,lin_ay,lin_az,grav_x,grav_y,grav_z,speed,lat,lon
2 1779785009399,2.0329502,9.61005,-0.89085007,-0.1500125,0.056925,-0.0479875,0.3071801,-1.4539251,1.8945813,0.841
3 1779785009399,1.149,8.094001,3.96795,-0.1500125,0.056925,-0.0479875,0.3071801,-1.4539251,1.8945813,0.84181994,5
4 1779785009405,-1.7310001,9.220051,6.6600003,-0.1500125,0.056925,-0.0479875,-2.562216,-0.32393932,4.5643525,0.83
5 1779785009412,0.060000002,10.6380005,1.8559501,0.1106875,0.022824999,0.01265,-2.562216,-0.32393932,4.5643525,0.83
6 1779785009421,1.01505,10.518001,-0.48795003,0.1203125,-0.0548625,0.0339625,-0.76513666,1.0924759,-0.23510623,0.8
7 1779785009421,0.89295006,8.66205,2.3840001,0.1203125,-0.0548625,0.0339625,-0.76513666,1.0924759,-0.23510623,0.8
8 1779785009430,2.2390501,0.5990505,3.2899501,-0.070125,-0.0218625,0.0306625,0.06321059,-0.8880682,0.23586702,0.8
9 1779785009430,0.5530504,10.450951,2.56005,0.090475,0.0526625,-0.0427625,-0.29406303,0.90695286,0.47081447,0.84
10 1779785234574,1.36695,8.976001,2.4859502,-0.1967625,-0.080575,0.089375,-0.23325787,-0.22755527,0.1550907,0.322
11 1779785234575,-0.675,10.288051,2.80095,-0.1967625,-0.080575,0.089375,-0.62885886,0.16098595,-0.9858351,0.32285
12 1779785234575,-0.30600002,9.675,1.36995,-0.1967625,-0.080575,0.089375,-0.62885886,0.16098595,-0.9858351,0.32285
13 1779785234588,0.31395,8.773951,4.63005,0.1139875,0.0204875,0.0191125,-0.018130928,-0.73616886,2.2688744,0.33208
14 1779785234590,0.88395005,9.8880005,1.02495,0.1139875,0.0204875,0.0191125,-0.018130928,-0.73616886,2.2688744,0.33208
15 1779785234599,0.165,9.568951,2.979,-0.045649998,0.0386375,-0.0033,0.5518733,0.37675858,-1.3407183,0.3339437,9.
16 1779785234609,0.057000004,9.93795,1.9759501,0.0374,-0.0231,0.056375,-0.16893436,0.0742359,0.6147387,0.33534795
17 1779785234620,-0.52395004,9.865951,2.67195,-0.04235,0.034375,-0.05115,-0.85442513,0.35448265,0.3069687,0.330475
18 1779785234630,1.3960501,9.20505,2.4469502,0.047025,-0.064625,0.056375,-0.85442513,0.35448265,0.3069687,0.330475
19 1779785234641,0.24705002,9.423,2.43495,-0.11055,0.013062499,-0.0160875,1.0584395,-0.306324,0.08259773,0.3376355
20 1779785234649,-1.19895,10.50405,0.67305005,0.2066625,0.0386375,-0.0724625,-0.09658592,-0.08683872,0.06443858,0.33703
21 1779785234660,0.57705003,8.08905,5.43195,0.1799875,0.0438625,-0.011825,0.24001676,-1.4180555,0.0504099,0.33703
22 1779785234674,0.57705003,10.289,0.75195,-0.0710075,0.0215875,0.0350625,0.23071236,0.76028347,-1.6231103,0.33703
23 1779785234679,0.33795002,9.505051,2.62905,-0.0718875,0.0215875,0.0350625,0.23071236,0.76028347,-1.6231103,0.33703
24 1779785234690,-0.13800001,9.972,2.466,0.044825,-0.031625,0.0424875,-0.001940817,-0.004078865,0.25600096,0.33972
25 1779785234700,-0.27495003,9.816001,2.08095,-0.0488125,0.0545875,-0.0776875,-0.47772253,0.46337795,0.09080589,0.33972
26 1779785234710,0.75195,9.61005,2.73,0.0672375,-0.097625,0.073425,-0.6073806,0.30708027,-0.29390465,0.34139836,9.
27 1779785234722,0.94395006,9.01395,2.26095,-0.1392875,0.0088,-0.0224125,0.41055167,0.100785255,0.3577156,0.34006
28 1779785234735,-1.8049501,10.812,0.78300005,0.2321,0.033275,-0.08415,0.6038871,-0.49313164,-0.120257616,0.336001
29 1779785234747,0.546,8.073,5.24205,-0.2872375,-0.0763125,0.0669625,-0.2076889,-1.4315748,2.8506103,0.3383111,9.56
30 1779785234755,1.1230501,9.967051,0.86895007,0.1023,-0.047575,0.0094875,0.2076889,-1.4315748,2.8506103,0.3455584
31 1779785234761,-0.41805002,9.8238505,2.8330503,-0.0499125,-0.0188375,-0.0383625,0.7774916,0.45876122,-1.5066352,0.3455584
32 1779785234780,-0.04695,9.55305,2.3730001,0.109725,-0.034925,0.0339625,-0.75678,0.31681728,0.4482634,0.3614386,9.
33 1779785234797,0.62505,9.543,2.3840001,-0.1722875,-0.031625,-0.0064625,-0.4083886,0.041514397,0.01282239,0.36088
34 1779785234800,-0.126,9.411,2.397,0.208725,0.0609125,-0.1192125,0.26419944,0.032050133,-0.058623314,0.36088057,5.
35 1779785234925,-0.89205,9.832951,1.07095,-0.1051875,-0.0433125,0.0339625,-0.48882852,-0.101638794,0.040277958,0.
```

Gambar IV.1 Contoh Hasil Data CSV

Berdasarkan hasil implementasi tersebut, aplikasi berhasil memenuhi kebutuhan penelitian sebagai media akuisisi data primer. Data yang dihasilkan memiliki struktur yang konsisten sehingga dapat diproses lebih lanjut pada tahapan *peak detection*, pelabelan data, pembentukan *event-centered window*, serta penyusunan dataset pelatihan sebagaimana dibahas pada subbab berikutnya.

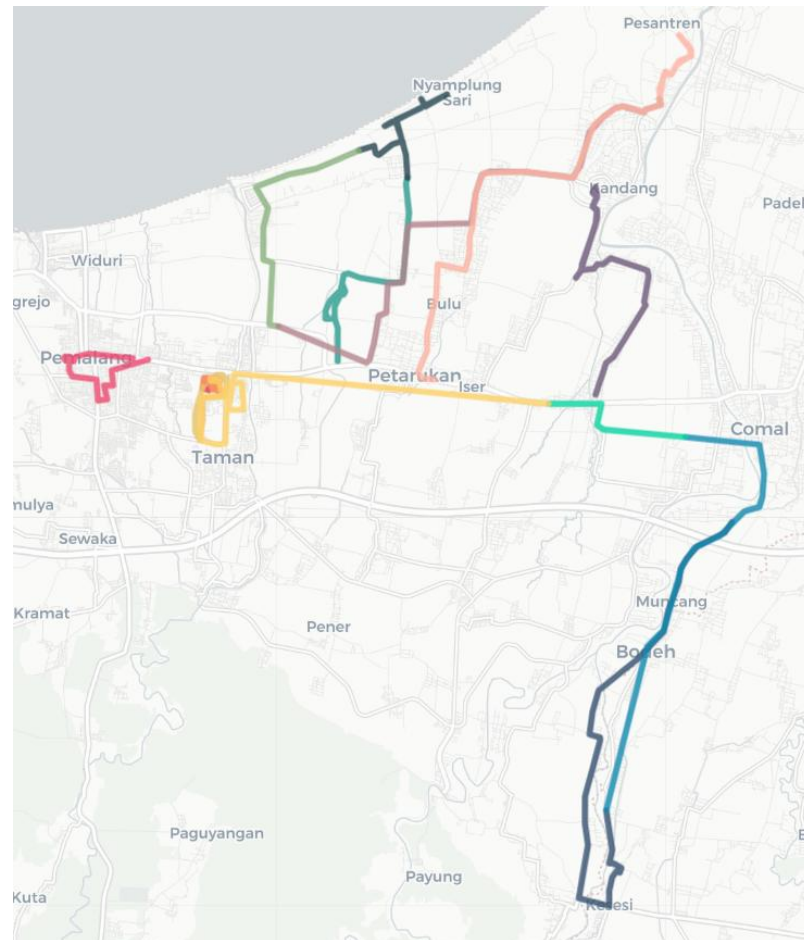
4.2 Pengumpulan dan Pembentukan Dataset

Subbab ini menyajikan hasil pengumpulan data lapangan hingga terbentuk dataset yang siap digunakan pada proses pelatihan model. Pembahasan meliputi hasil akuisisi data, pelabelan kandidat *event*, pembentukan dataset berbasis *event-centered window*, pembagian dataset menggunakan metode *Trip-Based Split*, serta normalisasi data sebelum proses pelatihan model.

4.2.1. Akuisisi Data Lapangan

Proses akuisisi data menghasilkan 16 perjalanan (*trip*) yang direkam menggunakan aplikasi *Data Logger* pada berbagai ruas jalan beraspal di Kabupaten Pemalang. Seluruh perjalanan dilakukan menggunakan sepeda

motor dengan smartphone yang dipasang pada *phone holder* di stang kendaraan sehingga posisi perangkat tetap konstan selama proses perekaman. Aplikasi merekam data akselerometer, giroskop, GPS, serta kecepatan kendaraan secara simultan pada berbagai ruas jalan beraspal di Kabupaten Pemalang.



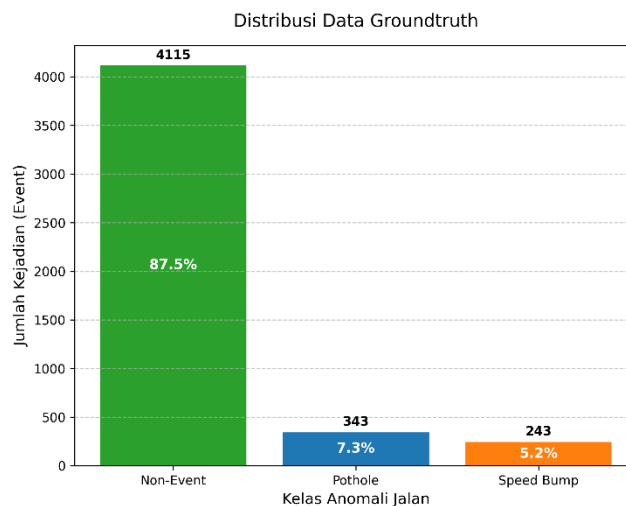
Gambar IV.2 Lintasan yang Ditempuh di Kabupaten Pemalang.

4.2.2. Pelabelan Data

Tahap berikutnya adalah proses pelabelan terhadap kandidat *event* yang diperoleh melalui proses *peak detection* pada sinyal akselerasi hasil tahap pra-pemrosesan sinyal. Sebelum dilakukan deteksi, data sensor terlebih dahulu diproses melalui *sensor fusion*, resampling menjadi 100 Hz, serta penyaringan menggunakan *Butterworth band-pass filter* sehingga menghasilkan sinyal *a_vertical* dan *a_linear_mag*. Peak detection kemudian diterapkan pada sinyal

a_linear_mag untuk memperoleh kandidat kejadian berdasarkan perubahan karakteristik getaran kendaraan yang melebihi ambang batas tertentu. Selanjutnya, setiap kandidat diverifikasi secara manual menggunakan antarmuka berbasis Folium dengan bantuan rekaman suara sebagai referensi waktu.

Melalui proses tersebut diperoleh 4.701 kandidat event yang kemudian diberikan label sesuai kondisi jalan sebenarnya. Rekaman suara tidak digunakan secara otomatis oleh sistem untuk melakukan sinkronisasi, melainkan hanya berfungsi sebagai petunjuk waktu (*time reference*) bagi peneliti dalam proses pelabelan manual.



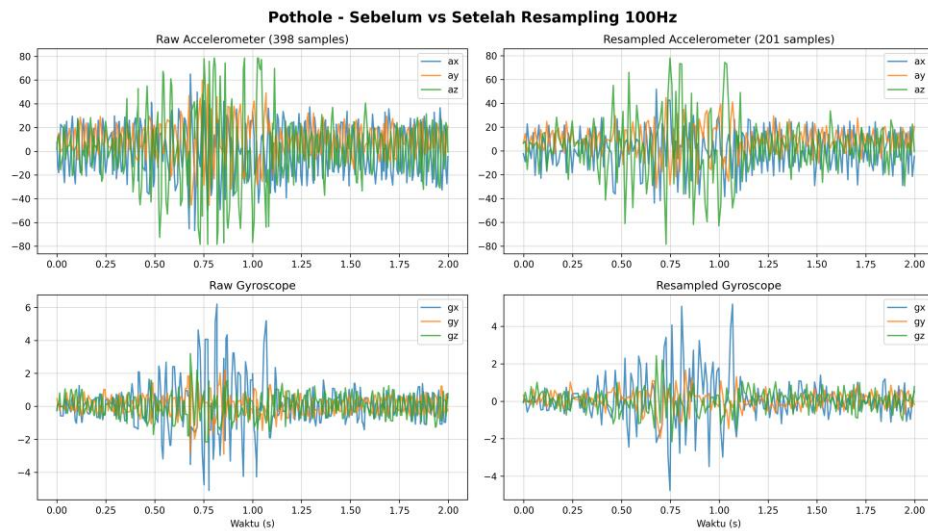
Gambar IV.3 Distribusi Data Hasil Pelabelan

Distribusi tersebut menunjukkan bahwa dataset memiliki karakteristik ketidakseimbangan kelas (*class imbalance*) yang cukup tinggi dengan rasio *Non-Event* terhadap kelas minoritas sekitar 12:1. Dominasi kelas *Non-Event* merupakan kondisi yang umum dijumpai pada sistem deteksi anomali karena sebagian besar ruas jalan yang dilalui berada dalam kondisi normal. Sebaliknya, kejadian *Pothole* maupun *Speed Bump* hanya muncul pada lokasi tertentu sehingga jumlah sampelnya jauh lebih sedikit. Karakteristik ini menjadi salah satu tantangan utama dalam proses pelatihan model dan menjadi dasar

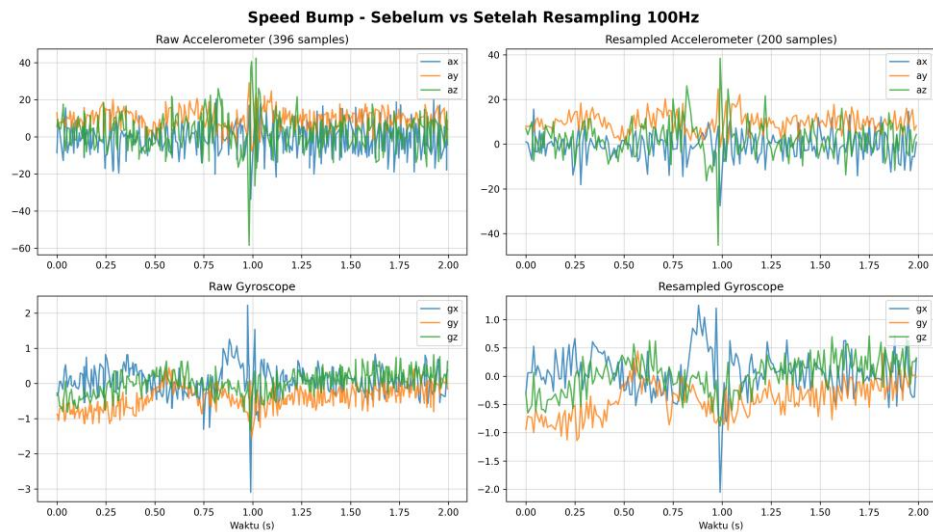
diterapkannya strategi penanganan *class imbalance* pada tahap pelatihan sebagaimana telah dijelaskan pada BAB III.

4.2.3. Pra-Pemrosesan Dataset

Seluruh kandidat *event* yang telah memperoleh label selanjutnya diproses menjadi dataset yang dapat digunakan sebagai masukan model. Tahap pertama dilakukan resampling terhadap data sensor menjadi frekuensi tetap 100 Hz menggunakan interpolasi linier sehingga interval waktu antar sampel menjadi seragam sebesar 10 ms.



Gambar IV.4 Resampling pada Kelas Pothole



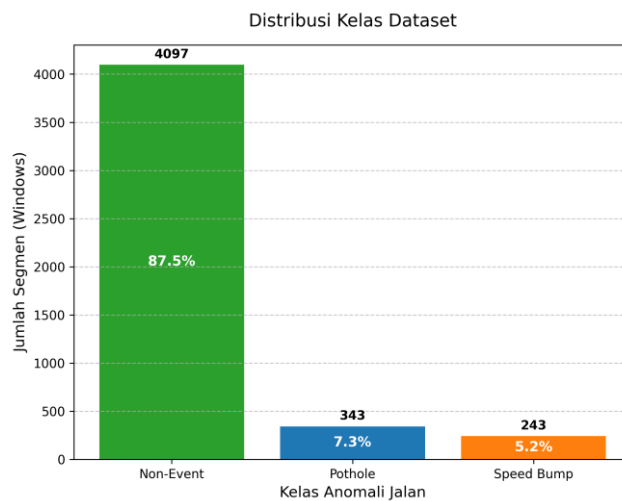
Gambar IV.5 Resampling pada Kelas Speed Bump

Setelah proses resampling selesai, dilakukan pembentukan event-centered window, yaitu pemotongan sinyal sepanjang 2,3 detik (230 timestep) dengan posisi kandidat event berada pada bagian tengah jendela. Window disimpan dengan panjang ekstra 30 timestep lebih panjang dari kebutuhan model untuk menyediakan ruang gerak bagi teknik augmentasi random crop pada tahap pelatihan, sebelum akhirnya dipotong menjadi 200 timestep (2 detik) tepat sebelum masuk ke model saat training maupun evaluasi. Window yang cakupan datanya kurang dari 70% akibat celah/data hilang, atau masih mengandung nilai kosong (NaN) setelah interpolasi, tidak diikutsertakan ke dalam dataset akhir.

Dari total 4.701 kandidat event, sebanyak 18 kandidat tidak berhasil dibentuk menjadi window dan dikeluarkan dari dataset akhir, dengan rincian 5 kandidat beririsan dengan jeda (gap) hilangnya sinyal sensor di tengah perjalanan, 6 kandidat gagal diekstraksi karena cakupan sinyal pada rentang window kurang dari 70% atau mengandung nilai kosong (NaN) setelah interpolasi dan 7 kandidat berada pada segmen kontinu yang berdurasi terlalu pendek untuk memenuhi ukuran window ekstensi yang dibutuhkan, umumnya karena berlokasi sangat dekat dengan titik awal atau akhir perjalanan. Dengan demikian, dataset akhir yang terbentuk berjumlah 4.683 window.

Tabel IV.1 Hasil Pembentukan Dataset

Parameter	Nilai
Kandidat <i>event</i> berlabel	4.701
<i>Window</i> akhir	4.683
Kanal <i>input</i>	7
Panjang <i>window</i> mentah	230 <i>time step</i>
Panjang <i>window</i> setelah crop	200 <i>time step</i>
Bentuk tensor CNN	[4.683, 7, 200]



Gambar IV.6 Distribusi Kelas Dataset

Hasil tersebut menunjukkan bahwa hampir seluruh kandidat *event* berhasil dipertahankan menjadi sampel pelatihan sehingga kehilangan data selama proses pembentukan dataset relatif kecil dan tidak memengaruhi representasi keseluruhan dataset.

4.3 Pelatihan Model

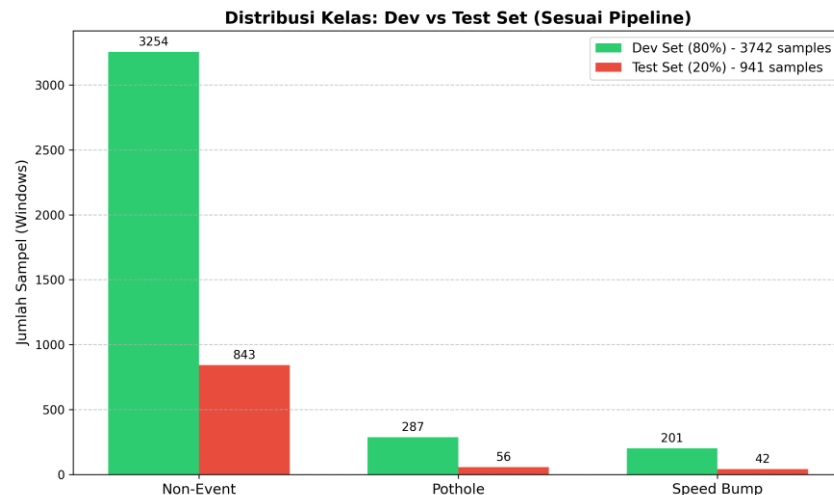
Subbab ini menyajikan hasil proses pelatihan model *InceptionTime1D* menggunakan dataset yang telah dibentuk pada subbab sebelumnya. Tahapan pelatihan meliputi pembagian dataset menggunakan metode *Trip-Based Split*, normalisasi data, konfigurasi proses pelatihan, optimasi *hyperparameter*, serta

pelatihan model hingga diperoleh model terbaik yang digunakan pada tahap evaluasi.

4.3.1. Pembagian Dataset

Setelah dataset akhir berhasil dibentuk sebanyak 4.683 window, tahap pertama pada proses pelatihan adalah melakukan pembagian dataset menggunakan metode *Trip-Based Split*. Berbeda dengan pembagian secara acak (*random split*), pendekatan ini mempertahankan seluruh sampel yang berasal dari satu perjalanan (*trip*) tetap berada pada subset yang sama sehingga mencegah terjadinya *data leakage* akibat kemiripan pola getaran antar sampel dalam perjalanan yang sama.

Dataset dibagi dengan rasio 80% sebagai Data Pelatihan (Training Set) dan 20% sebagai Data Pengujian (Holdout Test Set). Seluruh proses pelatihan model, optimasi *hyperparameter*, dan perhitungan parameter normalisasi hanya dilakukan menggunakan data pelatihan, sedangkan data pengujian diisolasi sepenuhnya hingga tahap evaluasi akhir. Hasil pembagian dataset disajikan pada Gambar 4.7.



Gambar IV.7 Hasil Pembagian Dataset menggunakan Metode Trip-Based Split.

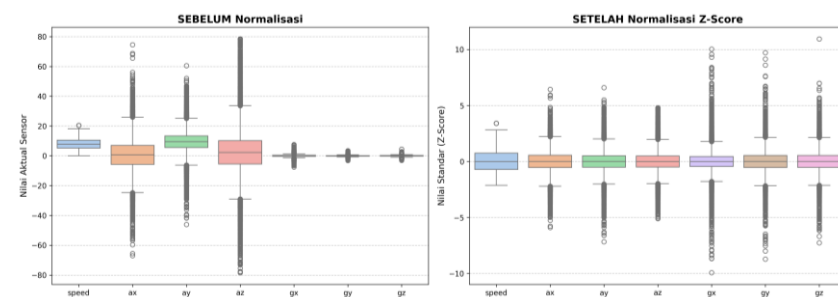
Karena rasio 80:20 diterapkan pada level trip (bukan level sampel window), dan jumlah window yang dihasilkan tiap trip tidak seragam, proporsi jumlah sampel akhir pada kedua subset dapat berbeda dari rasio nominal 80:20. Perlu juga diperiksa apakah distribusi kelas pada holdout test set proporsinya berbeda signifikan dari data pelatihan.

Pada proses pelatihan internal, data pengembangan (development set) selanjutnya dibagi lagi menggunakan StratifiedGroupKFold sebanyak empat lipatan untuk memperoleh data latih dan validasi secara terpisah berdasarkan grup perjalanan.

4.3.2. Normalisasi Dataset

Setelah proses pembagian dataset selesai dilakukan, tahap berikutnya adalah normalisasi data menggunakan metode *Global Z-Score Normalization*. Parameter normalisasi berupa nilai rata-rata (mean) dan simpangan baku (standard deviation) dihitung secara independen untuk setiap kanal, menggunakan seluruh Data Pelatihan (development set, 80%), kemudian digunakan untuk mentransformasikan seluruh data pelatihan maupun data pengujian.

Pendekatan ini diterapkan untuk mencegah terjadinya data leakage yang dapat muncul apabila parameter normalisasi dihitung menggunakan seluruh dataset (termasuk holdout test set). Selain itu, normalisasi dilakukan terhadap seluruh tujuh kanal masukan sehingga seluruh fitur memiliki skala numerik yang sebanding sebelum diproses oleh model.



Gambar IV.8 Hasil Global Z-Score Normalization

Hasil normalisasi menghasilkan distribusi data yang lebih seragam sehingga mempercepat proses konvergensi model sekaligus menjaga konsistensi skala antar kanal sensor.

4.3.3. Pelatihan Model

Model yang digunakan pada penelitian ini merupakan arsitektur InceptionTime1D dengan penambahan Squeeze-and-Excitation Block. Seluruh proses pelatihan dilakukan menggunakan framework PyTorch.

Untuk meningkatkan kemampuan model dalam mengenali kelas minoritas, diterapkan mekanisme augmentasi sinyal secara dinamis melalui DynamicJitterDataset, meliputi random crop (position jitter), time warping, Gaussian noise injection, magnitude scaling, dan channel dropout. Seluruh augmentasi hanya diterapkan pada Training Set, sedangkan data validasi dan data pengujian tetap digunakan dalam kondisi asli (center crop, tanpa deformasi tambahan).

Proses optimasi parameter model menggunakan optimizer Adam dengan learning rate awal, weight decay, batch size, dan lebar arsitektur (channels) yang seluruhnya ditentukan dari hasil pencarian hyperparameter, serta menerapkan gradient clipping dengan batas maksimum 1,0 untuk menjaga stabilitas proses optimasi.

Tabel IV.2 Konfigurasi Pelatihan Model

Parameter	Nilai
Arsitektur	InceptionTime1D + SE Block
Optimizer	Adam
Learning Rate Awal	0,00125
Scheduler	CosineAnnealingLR
Learning Rate Minimum	1×10^{-6}
Weight Decay	$1,62 \times 10^{-4}$
Batch Size	64

Dropout Rate	0,3441
Channels (lebar arsitektur)	64
Bottleneck Channels	16
Jumlah Epoch (fase tuning / cross-validation, maksimum)	50
Jumlah Epoch (model final)	33
Gradient Clipping	1,0
Loss Function	Multi-Class Focal Loss ($\gamma \approx 2,09$)
Augmentasi	DynamicJitterDataset

4.3.4. Optimasi Hyperparameter

Sebelum pelatihan akhir dilakukan, penelitian ini menerapkan optimasi hyperparameter menggunakan framework Optuna dengan sampler berbasis Tree-structured Parzen Estimator (TPE) dan MedianPruner (3 trial awal sebelum pruning aktif, 10 langkah pemanasan). Optimisasi dijalankan sebanyak 10 trial pada data pengembangan yang dibagi menggunakan StratifiedGroupKFold sebanyak 4 lipatan.

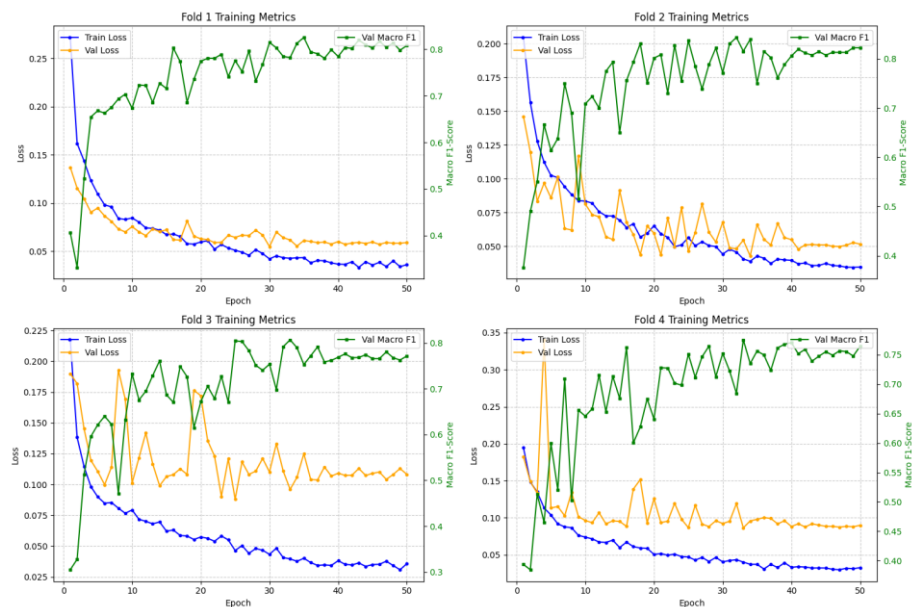
Parameter yang dioptimasi pada tahap ini meliputi *learning rate*, *dropout rate*, *batch size*, *weight decay*, *channels* atau lebar arsitektur, dan *gamma* pada Multi-Class Focal Loss. Sementara itu, jumlah epoch pencarian (50) serta seluruh parameter augmentasi sinyal ditetapkan tetap selama proses pencarian agar ruang pencarian hyperparameter tetap terkendali dan waktu komputasi tetap efisien.

Setiap kombinasi hyperparameter dilatih pada empat lipatan StratifiedGroupKFold cross-validation, kemudian nilai Macro F1-Score terbaik dari masing-masing lipatan dicatat. Nilai objektif yang dimaksimalkan bukan hanya rata-rata Macro F1-Score, melainkan juga robust score yang menghitung selisih antara rata-rata Macro F1-Score dan simpangan bakunya di antara 4 Fold (mean - standard deviation). Kombinasi hyperparameter terbaik hasil pencarian ini kemudian digunakan untuk mengonfigurasi skrip pelatihan model final.

4.3.5. Hasil Pelatihan Model

Model kemudian dilatih menggunakan konfigurasi terbaik hasil optimasi. Selama proses pelatihan pada skema cross-validation, dilakukan pemantauan terhadap training loss, validation loss, Macro F1-Score, serta PR-AUC pada data validasi, dengan bobot model dari epoch bernilai PR-AUC tertinggi disimpan sebagai *checkpoint* terbaik pada tiap lipatan (Best Model Checkpointing).

Perlu ditegaskan bahwa mekanisme Best Model Checkpointing tersebut diterapkan pada tahap validasi silang untuk menentukan jumlah epoch pelatihan yang optimal. Model final yang digunakan pada tahap evaluasi dan deployment selanjutnya dilatih ulang secara utuh (dari awal) menggunakan seluruh development set selama jumlah epoch tersebut, tanpa proses seleksi checkpoint lebih lanjut, karena tidak tersedia subset validasi terpisah pada tahap pelatihan model final ini, maka bobot yang disimpan adalah bobot dari epoch terakhir pelatihan.



Gambar IV.9 Kurva Train Loss, Val Loss dan Val Macro F1

Hasil pelatihan menunjukkan bahwa model mampu mencapai kondisi konvergen dengan penurunan *training loss* yang konsisten dan peningkatan

nilai *Macro F1-Score* selama proses optimasi. Model terbaik yang diperoleh pada tahap ini selanjutnya digunakan pada proses evaluasi performa.

4.4 Evaluasi Model

Subbab ini menyajikan hasil evaluasi performa model InceptionTime1D menggunakan Holdout Test Set yang sebelumnya diisolasi dari proses pelatihan maupun optimasi hyperparameter. Metrik evaluasi yang digunakan meliputi Accuracy, Precision, Recall, F1-Score, Macro F1-Score, Precision–Recall Area Under Curve (PR-AUC), serta Confusion Matrix

4.4.1. Hasil Evaluasi pada Holdout Test Set

Setelah proses pelatihan akhir selesai, model yang diperoleh pada akhir proses pelatihan dievaluasi menggunakan Holdout Test Set yang sepenuhnya dipisahkan dari proses pelatihan dan optimasi hiperparameter. Evaluasi ini bertujuan memperoleh estimasi performa model terhadap data perjalanan yang belum pernah diamati sebelumnya.

Hasil evaluasi menunjukkan bahwa model mampu mengklasifikasikan ketiga kelas kondisi jalan dengan baik. Ringkasan metrik evaluasi disajikan pada Gambar IV.10.

	precision	recall	f1-score	support
Non-Event	0.96	0.99	0.97	843
Pothole	0.85	0.62	0.72	56
Speed Bump	0.91	0.69	0.78	42
accuracy			0.95	941
macro avg	0.91	0.77	0.83	941
weighted avg	0.95	0.95	0.95	941

Gambar IV.10 Hasil Evaluasi Model Pada Holdout Test Set

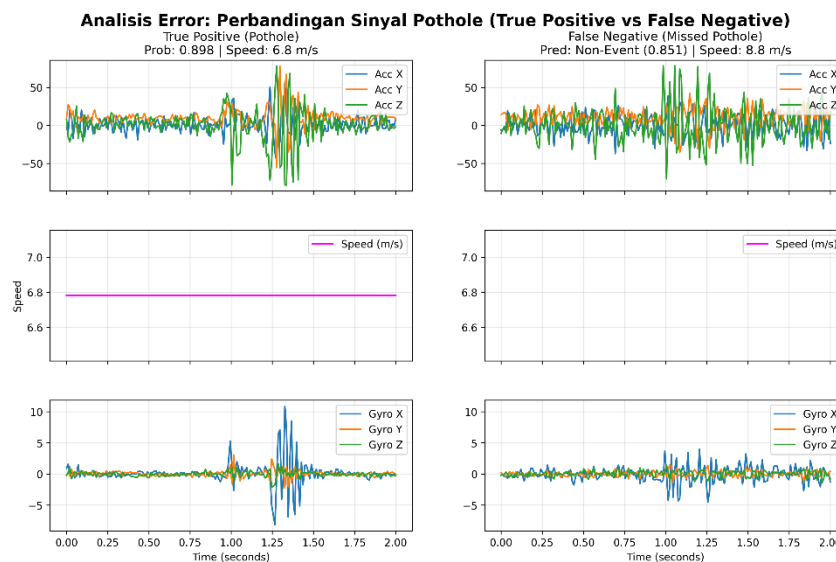
Pada penelitian ini, Macro F1-Score digunakan sebagai metrik utama karena lebih representatif untuk dataset dengan distribusi kelas yang tidak seimbang. Berbeda dengan *accuracy* yang cenderung didominasi oleh kelas Non-Event, Macro F1-Score memberikan bobot yang sama pada setiap kelas

sehingga kemampuan model dalam mendeteksi Pothole dan Speed Bump dapat dievaluasi secara lebih objektif.

Analisis per kelas menunjukkan bahwa Non-Event memperoleh performa terbaik dengan *precision* 0,96 dan *recall* 0,99. Hasil ini dipengaruhi oleh dominasi jumlah sampel serta karakteristik sinyal jalan normal yang relatif homogen sehingga lebih mudah dipelajari model.

Pada kelas Speed Bump, model mencapai *precision* 0,91 dan *recall* 0,69. Performa tersebut didukung oleh pola getaran polisi tidur yang relatif konsisten. Kesalahan klasifikasi umumnya terjadi pada polisi tidur dengan bentuk tidak standar atau ketika terdapat guncangan beruntun yang menghasilkan pola sinyal menyerupai kelas lain.

Sebaliknya, kelas Pothole menjadi kelas yang paling sulit dideteksi dengan nilai *recall* sebesar 0,62. Variasi ukuran, kedalaman, dan bentuk lubang jalan menghasilkan pola getaran yang sangat beragam sehingga meningkatkan kompleksitas klasifikasi. Untuk menganalisis kondisi tersebut, dilakukan perbandingan antara sinyal True Positive dan False Negative, sebagaimana ditunjukkan pada Gambar IV.11.



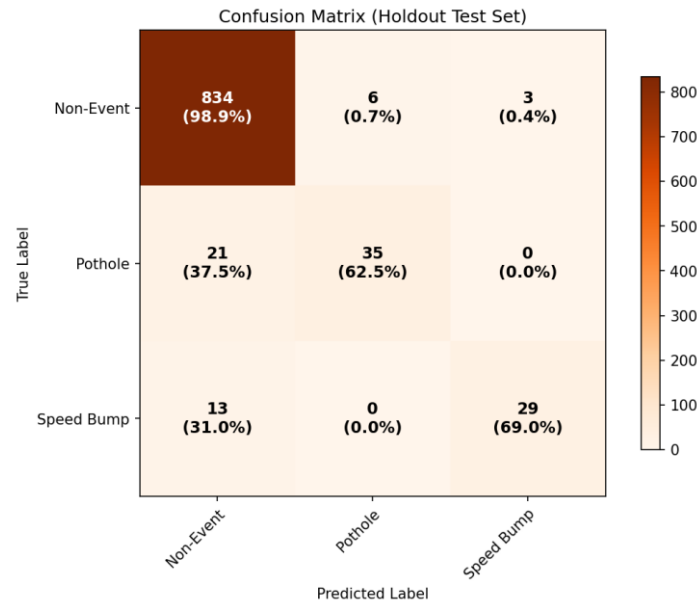
Gambar IV.11 Perbandingan Sinyal Pothole (TP vs FN)

Gambar IV.11 memperlihatkan bahwa False Negative umumnya berasal dari lubang jalan berukuran kecil atau yang dilalui pada kecepatan tinggi sehingga hanya menghasilkan lonjakan akselerasi vertikal tanpa adanya lonjakan rotasi dari gyroscope. Pola tersebut menyerupai sinyal Non-Event, menyebabkan model gagal mengenali anomali. Temuan ini menunjukkan bahwa variasi dinamika kendaraan masih menjadi salah satu keterbatasan penggunaan sensor smartphone dan membuka peluang pengembangan melalui normalisasi amplitudo berbasis kecepatan kendaraan.

Secara keseluruhan, model Lightweight InceptionTime 1D-CNN berhasil mempelajari representasi spasio-temporal sinyal IMU dengan baik. Kombinasi Inception Block, Squeeze-and-Excitation Block, dan Global Z-Score Normalization mampu menghasilkan performa yang seimbang pada dataset yang tidak seimbang, meskipun deteksi Pothole pada kondisi tertentu masih menjadi tantangan.

4.4.2. Analisis Confusion Matrix

Untuk memperoleh gambaran yang lebih rinci mengenai performa klasifikasi, dilakukan analisis menggunakan *Confusion Matrix*. Matriks ini menunjukkan jumlah prediksi benar maupun prediksi yang mengalami kesalahan pada setiap kelas.



Gambar IV.12 Confusion Matrix hasil evaluasi model pada Holdout Test Set.

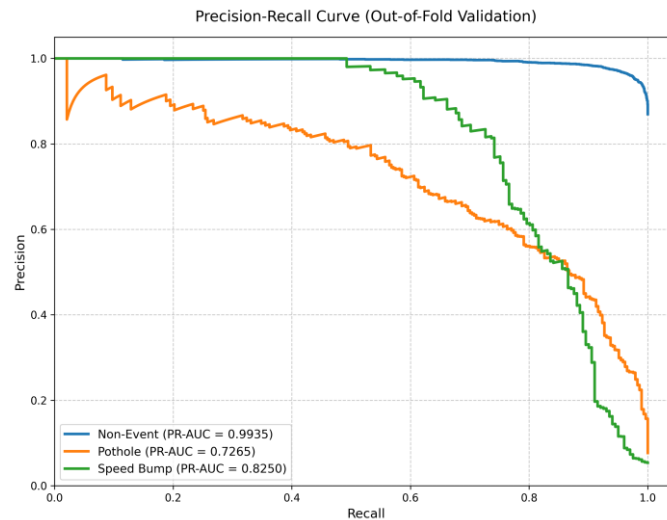
Berdasarkan Confusion Matrix, sebagian besar sampel *Non-Event* berhasil diklasifikasikan dengan benar. Hal tersebut menunjukkan bahwa model mampu membedakan pola getaran jalan normal dari pola getaran yang dihasilkan oleh anomali permukaan jalan.

Kesalahan klasifikasi masih ditemukan terutama antara kelas *Pothole* dan *Speed Bump*. Kondisi ini disebabkan kedua jenis anomali sama-sama menghasilkan impuls getaran vertikal yang relatif besar sehingga pada beberapa kondisi memiliki karakteristik temporal yang saling menyerupai. Selain itu, variasi kecepatan kendaraan dan karakteristik suspensi sepeda motor turut memengaruhi bentuk sinyal yang diterima sensor.

Meskipun demikian, jumlah kesalahan klasifikasi relatif kecil dibandingkan jumlah prediksi yang benar sehingga menunjukkan bahwa representasi fitur yang dipelajari oleh arsitektur *InceptionTimeID* telah mampu membedakan karakteristik utama ketiga kelas.

4.4.3. Analisis Precision–Recall Curve

Selain menggunakan metrik klasifikasi konvensional, penelitian ini juga mengevaluasi model menggunakan *Precision–Recall Curve* (PR Curve). Pendekatan ini dipilih karena lebih sesuai digunakan pada dataset yang memiliki distribusi kelas tidak seimbang.



Gambar IV.13 Kurva Precision–Recall untuk masing-masing kelas.

Hasil evaluasi menunjukkan bahwa kurva Precision–Recall memiliki luas area (PR-AUC) yang tinggi pada ketiga kelas. Nilai tersebut menunjukkan bahwa model mampu mempertahankan keseimbangan antara *precision* dan *recall* pada berbagai nilai probabilitas prediksi.

Pemilihan PR-AUC sebagai metrik utama juga konsisten dengan mekanisme *Best Model Checkpointing* yang digunakan selama proses pelatihan. Dengan demikian, model yang dipilih merupakan model dengan kemampuan terbaik dalam mendeteksi kelas minoritas tanpa terlalu dipengaruhi oleh dominasi kelas mayoritas.

4.5 Integrasi Edge AI (Deployment)

Setelah model InceptionTime1D terbaik diperoleh melalui proses pelatihan dan evaluasi, tahap berikutnya adalah mengintegrasikan model tersebut ke dalam

aplikasi Android agar dapat melakukan deteksi anomali permukaan jalan secara langsung (on-device inference).

4.5.1. Konversi Model PyTorch ke Format ONNX

Model hasil pelatihan disimpan dalam format checkpoint PyTorch (*.pth*) yang hanya dapat dijalankan menggunakan framework PyTorch. Agar model dapat digunakan pada perangkat Android, dilakukan proses konversi ke format *Open Neural Network Exchange* (ONNX).

Sebelum proses ekspor dilakukan, model dasar terlebih dahulu direkonstruksi menggunakan bobot hasil pelatihan **model final** (bukan hasil seleksi *best*). Selanjutnya dibangun sebuah *MobileInferenceWrapper* yang menggabungkan tiga komponen utama, yaitu parameter normalisasi Global Z-Score, model klasifikasi *InceptionTime1D*, dan fungsi *Softmax* sebagai keluaran probabilitas.

Dengan pendekatan ini, seluruh proses pra-pemrosesan yang diperlukan pada saat inferensi telah dibekukan (embedded) ke dalam grafik komputasi ONNX. Akibatnya, aplikasi Android tidak lagi perlu menghitung normalisasi maupun transformasi *Softmax* secara terpisah, tetapi cukup memberikan masukan berupa tensor hasil pembentukan window. Setelah proses wrapping selesai, model diekspor menggunakan fungsi `torch.onnx.export()`, kemudian disederhanakan menggunakan ONNX Simplifier (`onnxsim`).

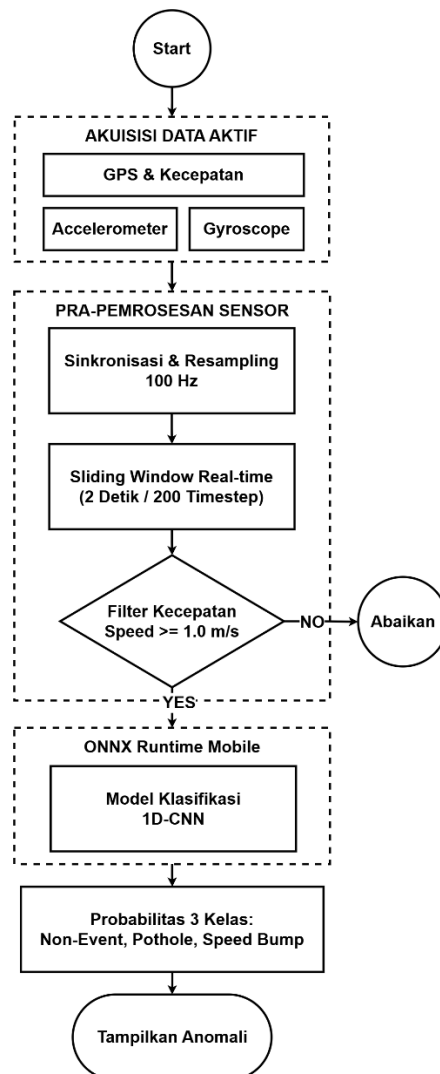
4.5.2. Implementasi ONNX Runtime pada Android

Model ONNX yang telah dihasilkan kemudian diintegrasikan ke dalam aplikasi Android menggunakan ONNX Runtime Mobile. Pada proses inferensi, aplikasi terlebih dahulu melakukan akuisisi data sensor akselerometer, giroskop, GPS, dan kecepatan kendaraan secara kontinu, kemudian membentuk event-centered window berdurasi 2 detik (200 timestep, hasil sliding window real-time, tanpa memerlukan buffer 230 timestep yang dipakai khusus untuk augmentasi crop saat pelatihan).

Sebagai mekanisme efisiensi tambahan, sistem menerapkan filter kecepatan pada tingkat aplikasi: proses inferensi model dilewati sepenuhnya

apabila kecepatan kendaraan (dari GPS) berada di bawah 1,0 m/s ($\sim 3,6$ km/jam), diimplementasikan pada kelas SensorFusionProcessor. Mekanisme ini mencegah kesalahan deteksi (false positive) akibat guncangan saat kendaraan diam atau bergerak sangat pelan, sekaligus menghemat konsumsi baterai.

Luaran model berupa probabilitas untuk tiga kelas, yaitu Non-Event, Pothole, dan Speed Bump. Hasil prediksi tersebut kemudian digunakan untuk menentukan jenis anomali yang ditampilkan pada antarmuka aplikasi beserta koordinat GPS lokasi kejadian.



Gambar IV.14 Pipeline Inferensi Model ONNX

Pendekatan ini membuat implementasi inferensi menjadi lebih sederhana karena seluruh logika klasifikasi berada di dalam model ONNX sehingga meminimalkan kemungkinan perbedaan hasil antara lingkungan pelatihan di Python dan implementasi pada Android.

4.5.3. Pengujian Fungsional Sistem

Setelah seluruh komponen sistem berhasil diintegrasikan, dilakukan pengujian fungsional menggunakan metode Black Box Testing untuk memastikan setiap fitur aplikasi bekerja sesuai dengan rancangan sistem.

Tabel 4.11 Hasil Pengujian Fungsional Aplikasi Android

No	Fitur yang Diuji	Hasil yang Diharapkan	Status
1	Memulai perekaman sensor	Sensor mulai merekam data	Berhasil
2	Menghentikan perekaman	Perekaman berhenti dan data tersimpan	Berhasil
3	Memuat model ONNX	Model berhasil dimuat	Berhasil
4	Melakukan inferensi	Prediksi kelas berhasil ditampilkan	Berhasil
5	Menampilkan lokasi pada peta	Marker muncul sesuai koordinat	Berhasil
6	Menyimpan hasil deteksi	Data tersimpan pada basis data lokal	Berhasil

Berdasarkan hasil pengujian tersebut, seluruh fungsi utama aplikasi dapat berjalan sesuai dengan rancangan. Model berhasil dimuat menggunakan ONNX Runtime Mobile dan mampu melakukan inferensi secara langsung pada perangkat Android tanpa memerlukan koneksi ke *cloud server*.

4.6 Pengujian Lapangan dan Pembahasan Implementasi (*Field Testing*)

Tahap akhir penelitian dilakukan melalui pengujian lapangan untuk mengevaluasi kinerja sistem secara langsung pada kondisi jalan yang sebenarnya. Pengujian ini bertujuan memverifikasi bahwa model yang telah dilatih tidak hanya memberikan performa yang baik pada pengujian *offline*, tetapi juga mampu bekerja secara konsisten ketika diimplementasikan pada perangkat Android.

4.6.1. Hasil Pengujian Lapangan

Pengujian lapangan dilakukan dengan mengendarai kendaraan roda dua pada berbagai ruas jalan beraspal di Kabupaten Pemalang sambil menjalankan aplikasi Android secara aktif. Hasil pengujian menunjukkan bahwa sistem mampu mendeteksi anomali permukaan jalan secara langsung selama perjalanan berlangsung, dengan lokasi yang diprediksi ditampilkan pada peta untuk memudahkan verifikasi terhadap kondisi jalan sebenarnya.

Selain itu, implementasi inferensi secara lokal memungkinkan proses klasifikasi dilakukan tanpa ketergantungan terhadap koneksi internet. Hal ini menunjukkan bahwa pendekatan *Edge AI* yang diterapkan berhasil memenuhi tujuan penelitian untuk menyediakan sistem deteksi anomali jalan yang dapat bekerja secara mandiri pada perangkat Android.

Gambar 4.18. Contoh hasil deteksi anomali

Gambar 4.19. Visualisasi Google Maps.

4.6.2. Pembahasan Implementasi Sistem

Hasil implementasi menunjukkan bahwa pendekatan Edge AI memberikan beberapa keuntungan dibandingkan pendekatan berbasis cloud: seluruh proses inferensi dilakukan secara lokal sehingga tidak memerlukan pengiriman data sensor ke server, berdampak pada berkurangnya latensi komunikasi sekaligus meningkatkan privasi data pengguna.

Integrasi MobileInferenceWrapper sebelum proses ekspor ONNX memberikan keuntungan dari sisi implementasi karena parameter normalisasi Global Z-Score dan fungsi Softmax telah menjadi bagian dari model ONNX,

sehingga hasil inferensi pada perangkat Android tetap konsisten dengan hasil yang diperoleh selama proses pelatihan dan evaluasi di lingkungan PyTorch.

Meskipun demikian, performa sistem di lapangan masih dipengaruhi oleh beberapa faktor eksternal, seperti variasi karakteristik suspensi kendaraan, perubahan kecepatan saat melintasi anomali, serta bentuk fisik kerusakan jalan yang sangat beragam.

Secara keseluruhan, hasil implementasi menunjukkan bahwa sistem yang dikembangkan telah mampu menjalankan proses deteksi anomali permukaan jalan secara *real-time* pada perangkat Android, mulai dari akuisisi data sensor, pembentukan *event-centered window*, inferensi menggunakan model ONNX, hingga visualisasi hasil deteksi pada peta digital.

BAB V

KESIMPULAN DAN PENELITIAN SELANJUTNYA

5.1 Kesimpulan

Berdasarkan hasil analisis, implementasi, dan evaluasi pengujian sistem deteksi anomali jalan menggunakan model 1D-CNN pada platform Android, dapat ditarik empat kesimpulan utama yang secara langsung menjawab rumusan masalah penelitian:

- 1 Sistem deteksi kerusakan jalan berbasis Edge AI berhasil dikembangkan dan mampu melakukan akuisisi serta pemrosesan data sensor smartphone secara real-time pada perangkat Android. Sistem terdiri atas dua fase, yaitu fase pertama sebagai aplikasi Data Logger yang digunakan untuk akuisisi data primer sebanyak 16 trip perjalanan, dan fase kedua yang mengintegrasikan akuisisi sensor (akselerometer, giroskop, dan GPS) melalui Foreground Service dengan proses inferensi model secara langsung pada perangkat (on-device inference) menggunakan ONNX Runtime Mobile. Hasil pengujian fungsional (Black Box Testing) menunjukkan seluruh 6 fitur utama aplikasi berjalan sesuai rancangan, dan hasil pengujian lapangan membuktikan sistem mampu melakukan deteksi anomali serta visualisasi lokasi pada peta digital tanpa ketergantungan pada koneksi internet.
- 2 Model Lightweight One-Dimensional Convolutional Neural Network (1D-CNN) dengan arsitektur InceptionTime multi-branch dan Squeeze-and-Excitation Block berhasil dibangun dan mampu mendeteksi tiga kelas kondisi jalan, yaitu Non-Event, Pothole, dan Speed Bump. Pada evaluasi holdout test set, model mencapai Macro F1-Score sebesar 0,83, dengan rincian performa per kelas yaitu Non-Event (precision 0,96; recall 0,99), Pothole (precision 0,85; recall 0,62), dan Speed Bump (precision 0,91; recall 0,69). Model kemudian berhasil dikonversi ke format ONNX melalui pembungkus MobileInferenceWrapper yang menyematkan parameter normalisasi Global Z-Score dan fungsi Softmax ke dalam graf model, sehingga proses inferensi pada perangkat Android dapat dilakukan secara

konsisten dengan hasil pelatihan di lingkungan PyTorch tanpa komputasi tambahan pada sisi aplikasi.

- 3 Kemampuan generalisasi model berhasil dievaluasi menggunakan strategi Trip-Based Holdout Split, yaitu pembagian dataset sebesar 80% development set dan 20% holdout test set berdasarkan lintasan perjalanan (trip), bukan berdasarkan jumlah sampel window. Pendekatan ini memastikan seluruh data pengujian berasal dari perjalanan yang sepenuhnya berbeda dari data pelatihan, sehingga risiko data leakage akibat kemiripan pola getaran dalam satu trip yang sama dapat dihindari. Development set selanjutnya dibagi menggunakan StratifiedGroupKFold sebanyak 4 fold untuk menjaga distribusi kelas tetap seimbang tanpa memecah satu trip ke lebih dari satu lipatan. Dengan strategi evaluasi ini, hasil performa model pada holdout test set (Macro F1-Score 0,83) diperoleh dari estimasi yang lebih merepresentasikan kondisi operasional di lapangan dibandingkan strategi random split konvensional.

5.2 Saran

Beberapa saran konkret untuk mengatasi kelemahan teknis sistem ini pada pengembangan selanjutnya adalah:

1. Normalisasi Sinyal Berbasis Kecepatan (Speed-Aware Scaling) Mengingat recall model pada kelas Pothole (62%) masih lebih rendah dibandingkan kelas lain, penelitian selanjutnya disarankan mengembangkan mekanisme penskalaan amplitudo sinyal akselerometer yang mempertimbangkan kecepatan kendaraan secara dinamis, guna mengurangi potensi false negative pada kecepatan rendah..
2. Validasi Spasial Berbasis Crowdsourcing Merancang mekanisme konsensus spasial pada sisi basis data lokal (Room DB) atau server, misalnya dengan memvalidasi suatu kejadian anomali hanya jika terdeteksi berulang pada beberapa perjalanan berbeda di koordinat yang sama, untuk menekan potensi false positive pada peta hasil deteksi.

3. Fusi dengan Data Visual (Computer Vision) Mengintegrasikan hasil deteksi berbasis IMU dengan model computer vision berbasis kamera untuk membedakan anomali sesungguhnya (pothole) dari sumber guncangan lain yang memiliki pola getaran serupa, seperti sambungan jalan atau rel kereta.
4. Pengukuran Kinerja Runtime secara Kuantitatif Penelitian selanjutnya disarankan melakukan pengukuran terstruktur terhadap metrik kinerja on-device seperti ukuran berkas model ONNX, waktu inferensi rata-rata, serta konsumsi daya baterai selama operasi berkelanjutan, karena aspek-aspek tersebut belum diukur secara kuantitatif pada penelitian ini namun penting untuk menilai kelayakan implementasi jangka panjang.

DAFTAR PUSTAKA

- Aguilar-gonzález, A., & Medina Santiago, A. (2025). CNN-Based Road Event Detection Using Multiaxial Vibration and Acceleration Signals. *Computer Science Department, Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE)*, 22–27.
- Aguilar-gonzález, A., & Medina Santiago, A. (2026). Context-Aware Road Event Detection Using Hybrid CNN – BiLSTM Networks. *Computer Science Department, Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE)*, 1–15.
- Arce-saenz, L. A., Izquierdo-reyes, J., & Bustamante-bello, R. (2025). *Exploring single-head and multi-head CNN and LSTM-based models for road surface classification using on-board vehicle multi-IMU data*. 1–19.
- Budi Darma, S., Kovacs, M., Serdult, U., & Kryssanov, V. (2022). ScienceDirect Semantic Segmentation on Smartphone Motion Sensor Data for Road Surface Monitoring. *Procedia Computer Science* 204, 00. <https://doi.org/10.1016/j.procs.2022.08.042>
- Capuano, V., Xu, L., & Benavides, J. E. (2023). Smartphone MEMS Accelerometer and Gyroscope Measurement Errors: Laboratory Testing and Analysis of the Effects on Positioning Performance. *Sensors*, 23(17).
- Ferrari, A., Micucci, D., Mobilio, M., & Napoletano, P. (2021). Trends in human activity recognition using smartphones. *Journal of Reliable Intelligent Environments*, 7(3), 189–213. <https://doi.org/10.1007/s40860-021-00147-0>
- Grouios, G., Ziagkas, E., Loukovitis, A., Chatzinikolaou, K., & Koidou, E. (2023). Accelerometers in Our Pocket: Does Smartphone Accelerometer Technology Provide Accurate Data? *Sensors*, 23(1), 1–17. <https://doi.org/10.3390/s23010192>
- Heydari, S., & Mahmoud, Q. H. (2025). Tiny Machine Learning and On-Device Inference: A Survey of Applications, Challenges, and Future Directions. *Sensors*, 25(10). <https://doi.org/10.3390/s25103191>

- Kim, G., & Kim, S. (2024). A Road Defect Detection System Using Smartphones. *Sensors*, 24(7). <https://doi.org/10.3390/s24072099>
- Kiranyaz, S., Avci, O., Abdeljaber, O., Ince, T., Gabbouj, M., & Inman, D. J. (2021). 1D convolutional neural networks and applications: A survey. *Mechanical Systems and Signal Processing*, 151, 107398. <https://doi.org/10.1016/j.ymssp.2020.107398>
- Koukaras, P., & Tjortjis, C. (2025). Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices. *Ai*, 6(10), 257. <https://doi.org/10.3390/ai6100257>
- Li, P., Wang, X., Huang, K., Huang, Y., Li, S., & Iqbal, M. (2022). Multi-Model Running Latency Optimization in an Edge Computing Paradigm. *Sensors*, 22(16). <https://doi.org/10.3390/s22166097>
- Opitz, J. (2024). A Closer Look at Classification Evaluation Metrics and a Critical Reflection of Common Evaluation Practice. *Transactions of the Association for Computational Linguistics*, 12(2018), 820–836. https://doi.org/10.1162/tacl_a_00675
- Ozoglu, F., & Gökgöz, T. (2023). Detection of Road Potholes by Applying Convolutional Neural Network Method Based on Road Vibration Data. *Sensors (Basel, Switzerland)*, 23(22). <https://doi.org/10.3390/s23229023>
- Richardson, E., Trevizani, R., Greenbaum, J. A., Carter, H., Nielsen, M., & Peters, B. (2024). The receiver operating characteristic curve accurately assesses imbalanced datasets. *Patterns*, 5(6), 100994. <https://doi.org/10.1016/j.patter.2024.100994>
- Safyari, Y., Mahdianpari, M., & Shiri, H. (2024). A Review of Vision-Based Pothole Detection Methods Using Computer Vision and Machine Learning. *Sensors*, 24(17). <https://doi.org/10.3390/s24175652>
- Utama, N. R., Arliansyah, J., & Kadarsah, E. (2023). Analisis Perbandingan Nilai Kondisi Jalan Menggunakan Metode SDI dan IRI Dari Alat Survei Roadroid. *Jurnal Penelitian Dan Kajian Bidang Teknik Sipil*, 12(1), 55–62. <https://doi.org/10.35139/cantilever.v12i1.233>

- Vergni, L., & Todisco, F. (2023). A Random Forest Machine Learning Approach for the Identification and Quantification of Erosive Events. *Water (Switzerland)*, 15(12). <https://doi.org/10.3390/w15122225>
- Zareei, M., Member, S., Alonzo, C., & Castañeda, L. (2025). *Machine Learning Model for Road Anomaly Detection Using Smartphone Accelerometer Data*. 13(July), 122841–122851.
- Zhao, K., Xu, S., Loney, J., Visentin, A., & Li, Z. (2024). Road pavement health monitoring system using smartphone sensing with a two-stage machine learning model. *Automation in Construction*, 167(February). <https://doi.org/10.1016/j.autcon.2024.105664>